Calcul Scientifique

Recherche de couples propres

Recherche de valeurs propres et vecteurs propres

Définition

$\lambda \in \mathbb{C}$ valeur propre de A : $\exists x \mathbb{C}^n,\ x \neq 0,\ A \cdot x = \lambda x$
x est appelé vecteur propre associé à λ.

Pourquoi chercher des valeurs propres (et des vecteurs propres) ?

- Résultats ayant une signification physique : mode de vibration d'une structure, traitement du signal, ...
- Eléments de réponse pour vérifier une propriété numérique : conditionnement d'une matrice, convergence de méthodes itératives, ...

Plusieurs types de problèmes

Pourquoi chercher des valeurs propres (et des vecteurs propres) ?

- Rechercher toutes les valeurs propres
  Exemple : les valeurs propres ont une signification physique
- Vérifier que les valeurs propres obéissent à une certaine propriété (le calcul exact n'est pas requis)
  Exemple : toutes les valeurs propres en module sont inférieures à 1
- Calculer la (les) plus grande(s) des valeurs propres en module et/ou la (les) plus petite(s), ainsi qu'une vecteur propre associé
  Exemple : calcul du nombre de conditionnement, algorithmes de classement de pages Web.

Plan

1. Localisation des valeurs propres
2. Algorithme de la puissance itérée / Cas d'une matrice symétrique $\Longrightarrow$ les valeurs propres sont obtenues successivement dans l'ordre décroissant de la valeur de leur module.
3. Algorithme de Jacobi / Cas d'une matrice symétrique $\Longrightarrow$ toutes les valeurs propres sont obtenues simultanément.

Localisation des valeurs propres

Théorème d'Hadamard-Gerchgörin

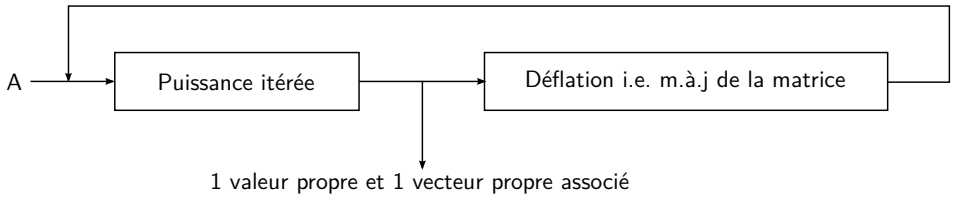
Soit $A \in \mathcal{M}_n(\mathbb{C})$, les valeurs propres de A ont des images dans le plan complexe qui appartiennent à $\bigcup_{i=1}^{n} D_i$ avec : $D_i = \left\{ z \in \mathbb{C} / |z - a_{ii}| \leqslant \sum_{j=1, j\neq i}^{n} |a_{ij}| \right\}$

Remarque : si $A \in \mathcal{M}_n(\mathbb{R})$ et si toutes les valeurs propres sont réelles, alors D_i et $\bigcup_{i=1}^{n} D_i$ sont des intervalles de $\mathbb{R}$

Corollaire au théorème d'Hadamard-Gerchgörin

$$\rho(A) \leqslant \max_{i=1,\ldots,n} \sum_{j=1}^{n} |a_{ij}| \qquad \rho(A) \leqslant \max_{j=1,\ldots,n} \sum_{i=1}^{n} |a_{ij}|$$

Algorithme de la puissance itérée



Principe

Hypothèse : toutes les valeurs propres sont réelles, non nulles et distinctes en module. Soient $|\lambda_1| > |\lambda_2| > \ldots > |\lambda_n|$ les valeurs propres de A

- 1ère application de l'algorithme $\Longrightarrow$ λ_1 et un vecteur propre associé
- Opération de déflation : modification de la matrice
- 2ème application de l'algorithme $\Longrightarrow$ λ_2 et un vecteur propre associé
- En n passages, toutes les valeurs propres et une base de vecteurs propres associés

Autres conditions d'application

- Si une valeur propre est complexe, l'algorithme échoue
- Si un sous-espace propre est de dimension supérieure à 1, obtention plusieurs fois de la même valeur propre, les vecteurs propres forment une base du sous-espace propre
- Procédé itératif convergent (suite de vecteurs) avec mise en œuvre d'un test d'arrêt et convergence pas toujours assurée quelque soit le vecteur de départ

Algorithme

Algorithm 1 Méthode de la puissance itérée (Power method)

Input : Matrice $A \in \mathcal{M}_n(\mathbb{R})$
Output : (λ_1, v_1) couple propre associé à la plus grande (en module) valeur propre.
$x_0 \in \mathbb{R}^n$ donné et $p = 0$
$\beta_p = {x_p}^T \cdot A \cdot x_p$
repeat
  $y_{p+1} = A \cdot x_p$
  $x_{p+1} = y_{p+1} / \|y_{p+1}\|$
  $\beta_{p+1} = {x_{p+1}}^T \cdot A \cdot x_{p+1}$
  $p = p + 1$
until $|\beta_{p+1} - \beta_p| / |\beta_p| < \varepsilon$
$\lambda_1 = \beta_{p+1}$ et $v_1 = x_{p+1}$

Convergence de la suite : $X_{p+1} = A \cdot X_p / \|A \cdot X_p\|$

Elements de preuve

$$\begin{cases} X_1 = \frac{1}{\|A \cdot X_0\|} A \cdot \left(\sum_{i=1}^{n} c_i V_i \right) ; \ \alpha_1 = \frac{1}{\|A \cdot X_0\|} ; \ X_1 = \alpha_1 \sum_{i=1}^{n} c_i \lambda_i V_i \\ X_p = \alpha_p \sum_{i=1}^{n} c_i \lambda_i^p V_i ; \ X_p = \alpha_p \lambda_1^p \left(c_1 V_1 + c_2 \left(\frac{\lambda_2}{\lambda_1} \right)^p V_2 + \ldots + c_n \left(\frac{\lambda_n}{\lambda_1} \right)^p V_n \right) \end{cases}$$

$$\left.\begin{array}{r} \lim_{p \to +\infty} \|X_{2p}\| = \lim_{p \to +\infty} \alpha_{2p} \lambda_1^{2p} \|c_1 V_1\| = 1 \\ \lim_{p \to +\infty} \alpha_{2p} \lambda_1^{2p} = \frac{1}{\|c_1 V_1\|} \end{array}\right\} \Longrightarrow \begin{array}{r} \lim_{p \to +\infty} X_{2p} = \frac{c_1 V_1}{\|c_1 V_1\|} = W_1 \\ \text{où } W_1 \text{ est un vecteur propre} \\ \text{normé associé à } \lambda_1 \end{array}$$

Méthode applicable si $c_1 \neq 0$ (faible taux d'échec avec choix arbitraire de X_0)

$$\begin{cases} \lim_{p \to +\infty} X_{2p+1} & = W_1 \text{ ou } -W_1 \text{ suivant le signe de } \lambda_1 \\ \lim_{p \to +\infty} \beta_p & = {W_1}^T . A . W_1 = \lambda_1 \end{cases}$$

Opération de déflation (Cas d'une matrice symétrique)

Principes

Soit $B = A - \lambda_1 W_1 \cdot {W_1}^T$

- Rang de $B = n - 1 \qquad (B \cdot W_1 = 0)$
- B est symétrique
- B possède les mêmes valeurs propres $\lambda_2, \ldots, \lambda_n$ que A et les mêmes vecteurs propres associés

L'application de l'algorithme de la puissance itérée à B produit λ_2 et W_2

$$C = B - \lambda_2 W_2 \cdot {W_2}^T \longrightarrow \lambda_3, W_3 \longrightarrow \ldots \longrightarrow \lambda_n, W_n$$

Exercice

1. Soit $A \in \mathcal{M}_n(\mathbb{R})$, λ valeur propre de A et u vecteur propre associé. Soit $\alpha \in \mathbb{R}$ non valeur propre de A. Montrer que $\mu = \frac{1}{\lambda - \alpha}$ est valeur propre de $(A - \alpha I)^{-1}$ et que, pour cette matrice, u est vecteur propre associé à μ.
2. Soit A symétrique et inversible. Soient $|\lambda_1| > |\lambda_2| > \ldots > |\lambda_n|$ les valeurs propres de A. On note $\| \cdot \|$ la norme vectorielle euclidienne.

   Quelles valeurs l'algorithme suivant permet-il d'obtenir ?

   i = 0, $x_i = \ldots$

   Boucler

   Résolution du système $A \cdot y_{i+1} = x_i$
   $x_{i+1} = \frac{y_{i+1}}{\|y_{i+1}\|}$
   $\beta_{i+1} = {x_{i+1}}^T \cdot A \cdot x_{i+1}$
   $i = i + 1$

   Jusqu'à convergence

Exercice

3. Quelles valeurs l'algorithme suivant permet-il d'obtenir ?

   i = 0, $x_i = \ldots$

   Boucler

   Résolution du système $(A - \alpha I) \cdot y_{i+1} = x_i$
   $x_{i+1} = \frac{y_{i+1}}{\|y_{i+1}\|}$
   $\beta_{i+1} = {x_{i+1}}^T \cdot A \cdot x_{i+1}$
   $i = i + 1$

   Jusqu'à convergence

Algorithme de Jacobi pour une matrice symétrique

Obtention simultanée de toutes les valeurs propres

Principes

Procédé itératif : $\begin{cases} A_1 = A \\ A_{k+1} = \Theta_k^{-1} \cdot A_k \cdot \Theta_k \end{cases}$

...jusqu'à la convergence : $\lim_{k \to +\infty} A_k = D = diag(\lambda_1, \ldots, \lambda_n)$

Choix de Θ_k ? Une matrice orthogonale ($\Theta_k^{-1} = {\Theta_k}^T$) car pour tout k :

- A_k est symétrique
- A_k possède les mêmes valeurs propres que A (vecteurs propres différents)

Pour obtenir les valeurs propres de A, il suffit donc que A_k converge vers une matrice diagonale.

Soient $A_k = \left[a_{ij}^{(k)}\right]$ et $S_k = \sum_{i=1}^{n}\sum_{j=1}^{n}\left(a_{ij}^{(k)}\right)^2 - \sum_{i=1}^{n}\left(a_{ii}^{(k)}\right)^2 \geqslant 0$

$\lim_{k \to +\infty} A_k = D \iff \lim_{k \to +\infty} S_k = 0$

Algorithme de Jacobi pour une matrice symétrique

Rotation de Givens

Θ_k choisie comme une matrice de rotation, i.e. définie à partir de 3 paramètres :

$$[i, j, \alpha] = [\text{numéro de ligne, numéro de colonne, angle de la rotation}]$$

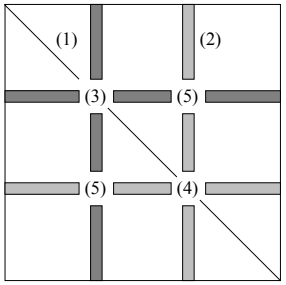
Soient $C = \cos(\alpha)$ et $S = \sin(\alpha)$

$$\Theta_k = \begin{array}{c} \\ \\ i \\ \\ j \\ \\ \end{array} \begin{array}{c} \begin{array}{cccccc} 1 & & & & & \\ & \ddots & & & & \\ & & C & \cdots & S & \\ & & \vdots & \ddots & \vdots & \\ & & -S & \cdots & C & \\ & & & & & \ddots \\ & & & & & & 1 \end{array} \\ \begin{array}{cccccc} & & i & & j & \end{array} \end{array}$$

Algorithme de Jacobi pour une matrice symétrique

Impact d'une itération

$A_{k+1} = {\Theta_k}^{-1} \cdot A_k \cdot \Theta_k$; quels sont les modifications entre k et $k+1$?



(1) $a_{ip}^{(k+1)} = a_{pi}^{(k+1)} = C \cdot a_{ip}^{(k)} - S \cdot a_{jp}^{(k)} \quad \forall p = 1, \ldots, n \qquad p \neq i, j$

(2) $a_{jp}^{(k+1)} = a_{pj}^{(k+1)} = S \cdot a_{ip}^{(k)} + C \cdot a_{jp}^{(k)} \quad \forall p = 1, \ldots, n \qquad p \neq i, j$

(3) $a_{ii}^{(k+1)} = C^2 \cdot a_{ii}^{(k)} - 2 \cdot C \cdot S \cdot a_{ij}^{(k)} + S^2 \cdot a_{jj}^{(k)}$

(4) $a_{jj}^{(k+1)} = S^2 \cdot a_{ii}^{(k)} + 2 \cdot C \cdot S \cdot a_{ij}^{(k)} + C^2 \cdot a_{jj}^{(k)}$

(5) $a_{ij}^{(k+1)} = a_{ji}^{(k+1)} = \left(C^2 - S^2\right) \cdot a_{ij}^{(k)} + C \cdot S \cdot \left(a_{ii}^{(k)} - a_{jj}^{(k)}\right)$

Algorithme de Jacobi pour une matrice symétrique

Choix des paramètres de la rotation

Des transformations précédentes :

$$S_{k+1} - S_k = 2\left(a_{ij}^{(k+1)}\right)^2 - 2\left(a_{ij}^{(k)}\right)^2$$

Pour la convergence, les paramètres sont fixés t.q. $S_{k+1} - S_k$ soit le plus négatif possible :

- On maximise $a_{ij}^{(k)} \implies$ valeur de i et j

$$\left|a_{ij}^{(k)}\right| = \max_{l,m=1,\ldots,n, l \neq m} \left|a_{lm}^{(k)}\right|$$

- On annule $a_{ij}^{(k+1)} \implies$ valeur de α

$$a_{ij}^{(k+1)} = \left(C^2 - S^2\right) \cdot a_{ij}^{(k)} + C \cdot S \cdot \left(a_{ii}^{(k)} - a_{jj}^{(k)}\right) = 0$$

  - Si $a_{jj}^{(k)} = a_{ii}^{(k)} \qquad \alpha = signe(a_{ij}^{(k)})\frac{\pi}{4}$
  - Si $a_{jj}^{(k)} \neq a_{ii}^{(k)} \qquad tg(2\alpha) = \frac{2a_{ij}^{(k)}}{a_{jj}^{(k)} - a_{ii}^{(k)}}$ avec $|\alpha| < \frac{\pi}{4}$

convergeance de la puissance itérée

$A \in S_n(\mathbb{R})$

Soit $(\lambda_i) \in \mathbb{R}^n$ les valeurs propres

$|\lambda_1| > |\lambda_2| > \dots > |\lambda_n|$

Soit $(v_i) \in (\mathbb{R}^n)^n$ une base de vecteurs propres associée

Soit $x_0 \in \mathbb{R}^n \setminus \{0\}$

$\exists ! (c_i) \in \mathbb{R}^n$ tq $x_0 = \sum_{i=1}^{n} c_i v_i$

- Itération 1 :

  - $y_1 = A x_0$

    $= \sum_{i=1}^{n} c_i \underbrace{A v_i}_{= \lambda_i v_i \text{ par déf de } \lambda_i \text{ et } v_i}$

    $= \sum_{i=1}^{n} c_i \lambda_i v_i$

  - $x_1 = \frac{y_1}{\|y_1\|_2} = \frac{\sum_{i=1}^{n} c_i \lambda_i v_i}{\| \sum_{i=1}^{n} c_i \lambda_i v_i \|_2}$

    $= \alpha_1 \sum_{i=1}^{n} c_i \lambda_i v_i$ avec $\alpha_1 > 0$

- Itération 2

  - $y_2 = \alpha_1 \sum_{i=1}^{n} c_i \lambda_i^2 v_i$

  - $x_2 = \alpha_2 \sum_{i=1}^{n} c_i \lambda_i^2 v_i$ avec $\alpha_2 > 0$

Donc $\forall p \in \mathbb{N}^*$

$x_p = \alpha_p \sum_{i=1}^{n} c_i \lambda_i^p v_i$ avec $\alpha_p > 0$

$x_p = \alpha_p \lambda_1^p \left(c_1 v_1 + \sum_{i=2}^{n} c_i \left(\frac{\lambda_i}{\lambda_1} \right)^p v_i \right)$

Hypothèse : $c_1 \neq 0$

En pratique, cette hypothèse ne pose pas de problème

d'où $\left| \frac{\lambda_i}{\lambda_1} \right| < 1 \quad \forall i \in [\![2, n]\!]$

et $\sum_{i=2}^{n} c_i \left(\frac{\lambda_i}{\lambda_1} \right)^p v_i \xrightarrow[p \to +\infty]{} 0$

Etude des ss suites paires et impaire

$* x_{2p} = \alpha_{2p} \lambda_1^{2p} \underbrace{\left(c_1 v_1 + \sum_{i=2}^{n} c_i \left(\frac{\lambda_i}{\lambda_1}\right)^{2p} v_i\right)}_{\xrightarrow[p\to+\infty]{} c_1 v_1}$

de plus $\|x_{2p}\|_2 = 1 \iff \alpha_{2p} \lambda_1^{2p} \left\| c_1 v_1 + \sum_{i=2}^{n} c_i \left(\frac{\lambda_i}{\lambda_1}\right)^{2p} v_i \right\|_2 = 1$

d'où $\alpha_{2p} \lambda_1^{2p} \xrightarrow[p\to+\infty]{} \frac{1}{\|c_1 v_1\|}$

et $x_{2p} \xrightarrow[p\to+\infty]{} \frac{c_1 v_1}{\|c_1 v_1\|} := w_1$ vecteur propre normalisé de A associé à λ_1.

$\beta_{2p} = x_{2p}^T A x_{2p}$

$\xrightarrow[p\to+\infty]{} w_1^T \underbrace{A w_1}_{= \lambda_1 w_1}$

$\xrightarrow[p\to+\infty]{} \lambda_1 \underbrace{\|w_1\|_2^2}_{=1}$

$\xrightarrow[p\to+\infty]{} \lambda_1$

$* x_{2p+1} = \alpha_{2p+1} \lambda_1^{2p+1} \underbrace{\left(c_1 v_1 + \sum_{i=2}^{n} c_i \left(\frac{\lambda_i}{\lambda_1}\right)^{2p+1} v_i\right)}_{\xrightarrow[p\to+\infty]{} c_1 v_1}$

De plus, $\|x_{2p+1}\|_2 = 1 \iff \alpha_{2p+1} \lambda_1^{2p} |\lambda_1| \left\| c_1 v_1 + \sum_{i=2}^{n} c_i \left(\frac{\lambda_i}{\lambda_1}\right)^{2p+1} v_i \right\|_2$
$= 1$

d'où $\alpha_{2p+1} \lambda_1^{2p} \xrightarrow[p\to+\infty]{} \frac{1}{|\lambda_1| \|c_1 v_1\|}$

et $x_{2p+1} \xrightarrow[p\to+\infty]{} \frac{\lambda_1 c_1 v_1}{|\lambda_1| \|c_1 v_1\|_2}$

$\xrightarrow[p\to+\infty]{} \underbrace{\frac{\lambda_1}{|\lambda_1|}}_{\text{signe}(\lambda_1)} w_1$

$\xrightarrow[p\to+\infty]{} \text{signe}(\lambda_1) w_1$

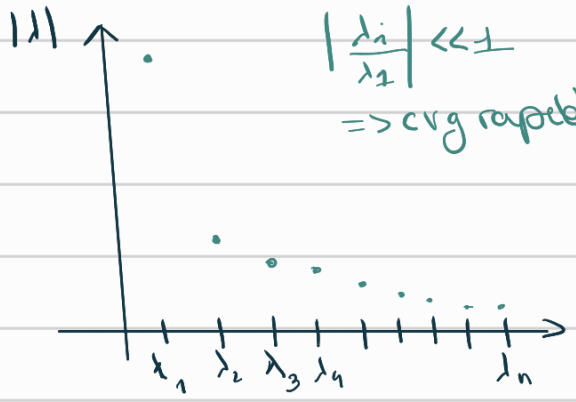
$\beta_{2p+1} = x_{2p+1}^T A x_{2p+1}$

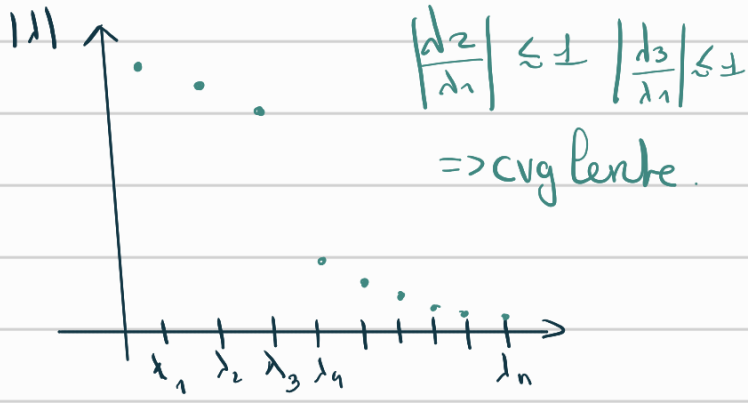
$$\xrightarrow[p\to+\infty]{} \underbrace{\text{signe}(\lambda_1)^2}_{=1}\,\lambda_1\,\underbrace{\|w_1\|_2^2}_{=1}$$

$$\xrightarrow[p\to+\infty]{} \lambda_1$$

Bilan : (β_p) converge vers λ_1
des ss-suites (x_{2p}) et (x_{2p+1}) convergent vers w_1 et $\text{signe}(\lambda_1)\,w_1$ qui sont des vecteurs propres normalisés associés à λ_1.

Vitesse de convergence





Exercice

2. (x_{2p}) et (x_{2p+1}) convergent vers des vecteurs propres de A^{-1} associé à la plus grande valeur propre de A^{-1} en module et donc l'inverse de la plus petite valeur propre de A en module, à savoir $\frac{1}{\lambda_n}$
$\Rightarrow (x_{2p})$ et (x_{2p+1}) convergent vers w_n et $\text{signe}(\lambda_n)w_n$
avec w_n vecteurs propre normalisé de A associé à λ_n
et $\beta_p \xrightarrow[p\to+\infty]{} w_n^T A w_n$
$\xrightarrow[p\to+\infty]{} \lambda_n$

1. $A \in \mathcal{M}_n(\mathbb{R})$
Soit $(\lambda, v) \in \mathbb{R} \times \mathbb{R}^n \setminus \{0\}$ couple propre de A
Mq : $\left(\frac{1}{\lambda-\alpha}, v\right)$ couple propre de $(A-\alpha I_n)^{-1}$

$A - \alpha I_n$ est inversible car α non valeur propre de A
$\Rightarrow \det(A - \alpha I_n) \neq 0$.

(λ, u) couple propre de $A \Leftrightarrow \begin{cases} Au = \lambda u \\ \text{et } u \neq 0 \end{cases}$

$\Leftrightarrow \begin{cases} Au - \alpha u = (\lambda - \alpha) u \\ \text{et } u \neq 0 \end{cases}$

$\Leftrightarrow \begin{cases} u \neq 0 \\ \text{et } (A - \alpha I_n) u = (\lambda - \alpha) u \end{cases}$

$\Leftrightarrow \begin{cases} u \neq 0 \\ \text{et } (A - \alpha I_n)^{-1} u = \frac{1}{\lambda - \alpha} u \end{cases}$

$\Leftrightarrow \left(\frac{1}{\lambda - \alpha}, u\right)$ couple propre de $(A - \alpha I_n)^{-1}$

3. 1. α valeur propre de A
$\Rightarrow$ algo "rank-revealing" : message d'erreur et base de $\ker(A - \alpha I_n)$

2. α non valeur propre de A
D'après 2., (x_{2p}) et (x_{2p+1}) convergent vers des vecteurs propres de A, associés à la plus grd valeur propre en valeur absolue de $(A - \alpha I_n)^{-1}$
mais $\max_{i \in [\![1,n]\!]} \frac{1}{|\lambda_i - \alpha|}$ (par $\perp$) $\Leftrightarrow \min_{i \in [\![1,n]\!]} |\lambda_i - \alpha|$

$\Leftrightarrow \lambda_{i_0}$ tq $|\lambda_{i_0} - \alpha| = \min_{i \in [\![1,n]\!]} |\lambda_i - \alpha|$

De plus (β_p) converge vers

Dense linear algebra : direct methods

P. Amestoy, P. Berger, M. Daydé, F.-H. Rouet
(INPT-ENSEEIHT)
and
J.-Y. L'Excellent (INRIA/LIP-ENS Lyon)

2021-2022

Outline

Linear Algebra Basics
- Gaussian Elimination and LU factorization
- LU Factorization with partial pivoting
- Symmetric matrices
- Cholesky Factorization

Linear Algebra Basics
- Gaussian Elimination and LU factorization
- LU Factorization with partial pivoting
- Symmetric matrices
- Cholesky Factorization

System of linear equations ?

Example:

$$\begin{array}{rcrcrcr} 2\,x_1 & - & 1\,x_2 & + & 3\,x_3 & = & 13 \\ -4x_1 & + & 6\,x_2 & - & 5\,x_3 & = & -28 \\ 6\,x_1 & + & 13\,x_2 & + & 16\,x_3 & = & 37 \end{array}$$

can be written under the form:

$$\mathbf{Ax} = \mathbf{b},$$

with $\mathbf{A} = \begin{pmatrix} 2 & -1 & 3 \\ -4 & 6 & -5 \\ 6 & 13 & 16 \end{pmatrix}$, $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$, and $\mathbf{b} = \begin{pmatrix} 13 \\ -28 \\ 37 \end{pmatrix}$

Gaussian Elimination

Example:

$$\begin{aligned} 2x_1 - x_2 + 3x_3 &= 13 && (1)\\ -4x_1 + 6x_2 - 5x_3 &= -28 && (2)\\ 6x_1 + 13x_2 + 16x_3 &= 37 && (3) \end{aligned}$$

With 2 * (1) + (2) $\rightarrow$ (2) and -3*(1) + (3) $\rightarrow$ (3) we obtain:

$$\begin{aligned} 2x_1 - x_2 + 3x_3 &= 13 && (4)\\ 0x_1 + 4x_2 + x_3 &= -2 && (5)\\ 0x_1 + 16x_2 + 7x_3 &= -2 && (6) \end{aligned}$$

Thus x_1 is eliminated from (5) and (6). With -4*(5) + (6) $\rightarrow$ (6):

$$\begin{aligned} 2x_1 - x_2 + 3x_3 &= 13\\ 0x_1 + 4x_2 + x_3 &= -2\\ 0x_1 + 0x_2 + 3x_3 &= 6 \end{aligned}$$

The linear system is then solved by backward ($x_3 \rightarrow x_2 \rightarrow x_1$) substitution: $x_3 = \frac{6}{3} = 2$, $x_2 = \frac{1}{4}(-2 - x_3) = -1$, and finally $x_1 = \frac{1}{2}(13 - 3x_3 + x_2) = 3$

LU Factorization

- Find **L** unit lower triangular and **U** upper triangular such that: $\mathbf{A} = \mathbf{L} \times \mathbf{U}$

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 3 \\ -4 & 6 & -5 \\ 6 & 13 & 16 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 3 & 4 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & -1 & 3 \\ 0 & 4 & 1 \\ 0 & 0 & 3 \end{bmatrix}$$

- Procedure to solve $\mathbf{Ax} = \mathbf{b}$
  - $\mathbf{A} = \mathbf{LU}$
  - Solve $\mathbf{Ly} = \mathbf{b}$ (*forward elimination*, down)
  - Solve $\mathbf{Ux} = \mathbf{y}$ (*backward substitution*, up)

$\mathbf{Ax} = (\mathbf{LU})\mathbf{x} = \mathbf{L}(\mathbf{Ux}) = \mathbf{Ly} = \mathbf{b}$

From Gaussian Elimination to LU Factorization

$\mathbf{A} = \mathbf{A}^{(1)}$, $\mathbf{b} = \mathbf{b}^{(1)}$, $\mathbf{A}^{(1)}\mathbf{x} = \mathbf{b}^{(1)}$:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} \quad \begin{aligned} 2 &\leftarrow 2 - 1 \times a_{21}/a_{11} \\ 3 &\leftarrow 3 - 1 \times a_{31}/a_{11} \end{aligned}$$

$\mathbf{A}^{(2)}\mathbf{x} = \mathbf{b}^{(2)}$

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} \\ 0 & a_{32}^{(2)} & a_{33}^{(2)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2^{(2)} \\ b_3^{(2)} \end{pmatrix} \quad \begin{aligned} b_2^{(2)} &= b_2 - a_{21} b_1/a_{11} \\ a_{32}^{(2)} &= a_{32} - a_{31} a_{12}/a_{11} \end{aligned}$$

Finally $3 \leftarrow 3 - 2 \times a_{32}/a_{22}$ gives $\mathbf{A}^{(3)}\mathbf{x} = \mathbf{b}^{(3)}$

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} \\ 0 & 0 & a_{33}^{(3)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2^{(2)} \\ b_3^{(3)} \end{pmatrix} \quad a_{33}^{(3)} = a_{33}^{(2)} - a_{32}^{(2)} a_{23}^{(2)}/a_{22}^{(2)}$$

From Gaussian Elimination to LU Factorization

Typical Gaussian elimination at step k:

$$\begin{pmatrix} a_{11}^{(1)} & \dots & \dots & \dots & \dots & \dots & a_{1n}^{(1)} \\ 0 & \ddots & & & & & \vdots \\ \vdots & \ddots & a_{k-1k-1}^{(k-1)} & \dots & \dots & \dots & a_{k-1n}^{(k-1)} \\ \vdots & & 0 & a_{kk}^{(k)} & a_{kk+1}^{(k)} & \dots & a_{kn}^{(k)} \\ \vdots & & \vdots & a_{k+1k}^{(k)} & a_{k+1k+1}^{(k)} & \dots & a_{k+1n}^{(k)} \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ 0 & \dots & 0 & a_{nk}^{(k)} & a_{nk+1}^{(k)} & \dots & a_{nn}^{(k)} \end{pmatrix} \quad \begin{pmatrix} b_1^{(1)} \\ \vdots \\ b_{k-1}^{(k-1)} \\ b_k^{(k)} \\ b_{k+1}^{(k)} \\ \vdots \\ b_n^{(k)} \end{pmatrix}$$

$$\boxed{a_{ij}^{(k+1)} = a_{ij}^{(k)} - \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}} a_{kj}^{(k)}, \text{ for } i > k}$$

(and $a_{ij}^{(k+1)} = a_{ij}^{(k)}$ for $i \leq k$)

From Gaussian Elimination to **LU** factorization

$$\begin{cases} a_{ij}^{(k+1)} = a_{ij}^{(k)} - \dfrac{a_{ik}^{(k)}}{a_{kk}^{(k)}} a_{kj}^{(k)}, \text{ for } i > k \\ a_{ij}^{(k+1)} = a_{ij}^{(k)}, \text{ for } i \leq k \end{cases}$$

- One step of Gaussian elimination can be written: $\mathbf{A}^{(k+1)} = \mathbf{L}^{(k)}\mathbf{A}^{(k)}$ (and $b^{(k+1)} = \mathbf{L}^{(k)}b^{(k)}$), with

$$\mathbf{L}^k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{k+1,k} & \ddots & \\ & & \vdots & & \\ & & -l_{n,k} & & 1 \end{pmatrix} \text{ and } l_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}.$$

- After $n-1$ steps, $\mathbf{A}^{(n)} = \mathbf{U} = \mathbf{L}^{(n-1)} \dots \mathbf{L}^{(1)}\mathbf{A}$ gives $\boxed{\mathbf{A} = \mathbf{LU}}$, with $\mathbf{L} = [\mathbf{L}^{(1)}]^{-1} \dots [\mathbf{L}^{(n-1)}]^{-1} =$

$$\begin{pmatrix} 1 & & & \\ l_{21} & 1 & & \\ \vdots & & \ddots & \\ l_{n1} & & & 1 \end{pmatrix} \dots \begin{pmatrix} 1 & & & \\ & \ddots & & \\ & & 1 & \\ & & l_{n,n-1} & 1 \end{pmatrix} = \begin{pmatrix} 1 & & 0 & \\ l_{21} & 1 & & \\ \vdots & & \ddots & \\ l_{n1} & \dots & l_{n,n-1} & 1 \end{pmatrix}$$

- Avoid building the zeros under the diagonal
- Before

```
L(n,n)=1
do k=1, n-1
   L(k,k) = 1
   do i=k+1, n
      L(i,k) = A(i,k)/A(k,k)
      do j=k, n
         A(i,j) = A(i,j) - L(i,k) * A(k,j)
```

- After

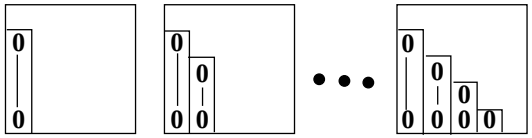
```
L(n,n)=1
do k=1, n-1
   L(k,k) = 1
   do i=k+1, n
      L(i,k) = A(i,k)/A(k,k)
      do j=k+1, n
         A(i,j) = A(i,j) - L(i,k) * A(k,j)
```


LU Factorization Algorithm

- Overwrite matrix **A**: we store $a_{ij}^{(k)}$, $k = 2, \dots, n$ in `A(i,j)`
- In the end, $\mathbf{A} = \mathbf{A}^{(n)} = \mathbf{U}$

```
do k=1, n-1
   L(k,k) = 1
   do i=k+1, n
      L(i,k) = A(i,k)/A(k,k)
      do j=k, n  ! (better than: do j=1,n)
         A(i,j) = A(i,j) - L(i,k) * A(k,j)
      end do
   enddo
enddo
L(n,n)=1
```

- Matrix **A** at each step:



- Use lower triangle of array **A** to store $\mathbf{L}_{i,k}$ multipliers
- Before:

```
L(n,n)=1
do k=1, n-1
   L(k,k) = 1
   do i=k+1, n
      L(i,k) = A(i,k)/A(k,k)
      do j=k+1, n
         A(i,j) = A(i,j) - L(i,k) * A(k,j)
```

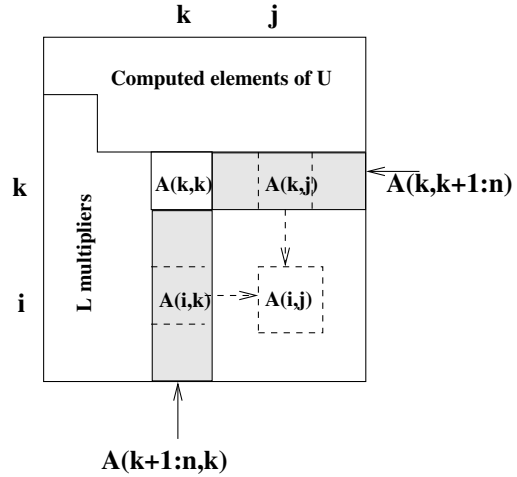
- After (diagonal 1 of L is not stored):

```
do k=1, n-1
   do i=k+1, n
      A(i,k) = A(i,k)/A(k,k)
      do j=k+1, n
         A(i,j) = A(i,j) - A(i,k) * A(k,j)
```

- More compact array syntax (Matlab, scilab):

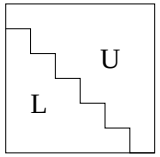
```
do k=1, n-1
  A(k+1:n,k) = A(k+1:n,k) / A(k,k)
  A(k+1:n,k+1:n) = A(k+1:n,k+1:n)
                   - A(k+1:n,k) * A(k,k+1:n)
end do
```

- corresponds to a rank-1 update:



What we have computed

- we have stored the **L** and **U** factors in **A**:
  - $\mathbf{A}_{i,j}$, i > j corresponds to l_{ij}
  - $\mathbf{A}_{i,j}$, i $\leq$ j corresponds to u_{ij}
  - with $l_{ii} = 1, i = 1, n$
- Finally,



after factorization: $\mathbf{A} = \mathbf{L} + \mathbf{U} - I$

LU factorization : summary

- Step by step columns of **A** are set to zero and **A** is updated $\mathbf{L}^{(n-1)} \dots \mathbf{L}^{(1)}\mathbf{A} = \mathbf{U}$ leading to $\mathbf{A} = \mathbf{LU}$ where $\mathbf{L} = [\mathbf{L}^{(1)}]^{-1} \dots [\mathbf{L}^{(n-1)}]^{-1}$
- At each step $\mathbf{A}(k,k)$ is referred to as the pivot
  - zero entries in column of **A** can be replaced by entries in **L**
  - row entries of **U** are stored in corresponding locations of **A**

Algorithm 1 **LU** factorization

```
for k = 1, n-1 do
  if |A(k,k)| too small then
    exit (small pivots are not allowed)
  end if
  A(k+1:n,k) = A(k+1:n,k) / A(k,k)
  A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - A(k+1:n,k)*A(k,k+1:n)
end for
```

When $|\mathbf{A}(k,k)|$ is too small, one could consider other pivots: numerical pivoting strategies will be introduced later.

Existence and uniqueness of **LU** decomposition

Theorem 1

*$\mathbf{A} \in R^{n\times n}$ has an LU factorization (where **L** is unit lower triangular and **U** is upper triangular) if $det(\mathbf{A}(1:k,1:k)) \neq 0$ for all $k \in \{1 \dots n-1\}$. If the **LU** factorization exists, then it is unique and $det(\mathbf{A}) = u_{11} \dots u_{nn}$.*

Theorem 2

*For each nonsingular matrix **A**, there exists a permutation matrix **P** such that **PA** possesses an LU factorization **PA** = **LU**.*

Definition 1

$\mathbf{A} \in R^{n\times n}$ is strictly diagonally dominant iff $|a_{ii}| > \sum_{j=1,j\neq i}^{n} |a_{ij}|$ for all $i = 1, \dots, n$

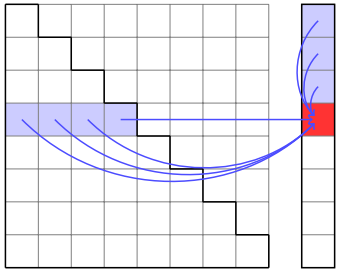
Theorem 3

*If $\mathbf{A}^T$ is strictly diagonally dominant then **A** is non-singular and **A** has an LU factorization and $l_{ij} \leq 1$*

Solution phase : $\mathbf{Lx} = \mathbf{b}$ (Left-Looking and Right-looking)

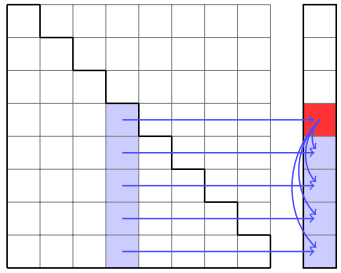
Algorithm 2 LL (sans report)

$x = b$
for $j = 1,\ n$ **do**
  for $i = 1,\ j-1$ **do**
    $x_j = x_j - l_{ji}x_i$
  end for
  $x_j = \frac{x_j}{l_{jj}}$
end for



Algorithm 3 RL (avec report)

$x = b$
for $j = 1,\ n$ **do**
  $x_j = \frac{x_j}{l_{jj}}$
  for $i = j+1,\ n$ **do**
    $x_i = x_i - l_{ij}x_j$
  end for
end for



Blocked **LU** and Schur decomposition

Exercise 1 (Blocked **LU** and Schur decomposition)

Let $\mathbf{A}$ *be a non singular matrix of order n for which* $\exists \mathbf{P}$ *permutation matrix such that* $\mathbf{PA}$ *can be factored without pivoting and consider the block form* $\mathbf{PA} = \begin{pmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} \end{pmatrix}$. *We define the so called Schur matrix* $\mathbf{S}$ *as*

$\mathbf{S} = \mathbf{A}_{2,2} - \mathbf{A}_{2,1}\left(\mathbf{A}_{1,1}\right)^{-1}\mathbf{A}_{1,2}$

1. *Explain how to adapt the* **LU** *factorisation algorithm to obtain the following decomposition of* $\mathbf{PA}$. $\mathbf{PA} = \begin{pmatrix} \mathbf{L}_{1,1} & 0 \\ \mathbf{L}_{2,1} & I \end{pmatrix}\begin{pmatrix} \mathbf{U}_{1,1} & \mathbf{U}_{1,2} \\ 0 & \mathbf{S} \end{pmatrix} = \begin{pmatrix} \mathbf{L}_{1,1} & 0 \\ \mathbf{L}_{2,1} & I \end{pmatrix}\begin{pmatrix} I & 0 \\ 0 & \mathbf{S} \end{pmatrix}\begin{pmatrix} \mathbf{U}_{1,1} & \mathbf{U}_{1,2} \\ 0 & I \end{pmatrix}$
2. *Prove that* $det(\mathbf{A}) = det(\mathbf{P})det(\mathbf{A}_{1,1})det(\mathbf{S})$
3. *We assume that we know how to compute* $\mathbf{Y}$ *such that* $\mathbf{Y} = \mathbf{S}^{-1}\mathbf{Z}$. *Describe how to use previous incomplete blocked factorization to solve* $\mathbf{AX} = \mathbf{B}$.

Blocked factorization and null space

Exercise 2 (Null space)

We suppose that after $n - r$ *steps of* **LU** *factorization we have* $\mathbf{PA} = \begin{pmatrix} \mathbf{L}_{1,1} & 0 \\ \mathbf{L}_{2,1} & I_r \end{pmatrix}\begin{pmatrix} \mathbf{U}_{1,1} & \mathbf{U}_{1,2} \\ 0 & \mathbf{S}_r \end{pmatrix}$ *where* $\mathbf{S}_r$ *is the Schur complement matrix of order r. We also suppose that* $\mathbf{S}_r = 0$ *(in practice one could also assume that* $\|\mathbf{S}_r\| \leq \varepsilon\|\mathbf{A}\|$ *for some matrix norm). Finally we assume that* $det(\mathbf{U}_{11}) \neq 0$.
Prove that the dimension of the null-space is r and compute a basis of the null space.

Number of floating-point operations (flops)

- In forward elimination ($Ly = b$), computing the kth unknown

$$y_k = b_k - \sum_{j=1}^{k-1} L_{kj}y_j$$

  leads to (k-1) multiplications and (k-1) additions, for $1 \leq k \leq n$
  $n^2 - n$ flops overall
- Idem for $Ux = y$ and at worst n divisions ($U_{kk} \neq 1$).
- Number of flops during factorization:
  - $n - k$ divisions
  - $(n-k)^2$ multiplications, $(n-k)^2$ additions
  - $k = 1, 2, ..., n-1$
  - total: $\approx \frac{2\times n^3}{3}$
    (Strassen's algorithm can reduce this to $\Theta(n^{log_2 7}) \simeq \Theta(n^{2.8})$)

Exercise 3 (How to compute $\mathbf{x}$ such that $\mathbf{x} = (\mathbf{A}^2)^{-1}\mathbf{b}$)

Let $\mathbf{A}$ be a non singular matrix of order n (i.e. it exists $\mathbf{L}$, $\mathbf{U}$ and $\mathbf{P}$ such that $\mathbf{PA} = \mathbf{LU}$ (note that $\mathbf{A}^2$ is also non singular).

1. *Compare the computational complexity of solving $\mathbf{A}^2\mathbf{x} = \mathbf{b}$ with the following two algorithms:*
   - 1.1 *Compute $\mathbf{B} = \mathbf{A}^2$, factor $\mathbf{B}$ and solve $\mathbf{Bx} = \mathbf{b}$*
   - 1.2 *Factor $\mathbf{A}$ and use the factored form to solve $\mathbf{A}^2\mathbf{x} = \mathbf{b}$*
2. *Explain why computing directly $\mathbf{C} = (\mathbf{A}^2)^{-1}$ and performing $\mathbf{x} = \mathbf{Cb}$ is not a method of choice.*

Linear Algebra Basics
Gaussian Elimination and LU factorization
LU Factorization with partial pivoting
Symmetric matrices
Cholesky Factorization

Consider $A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \frac{1}{\varepsilon} & 1 \end{bmatrix} \times \begin{bmatrix} \varepsilon & 1 \\ 0 & 1 - \frac{1}{\varepsilon} \end{bmatrix}$

$$\kappa_2(A) = \frac{\lambda_{max}}{\lambda_{min}} = \frac{1 + \varepsilon + \sqrt{5 + \varepsilon^2 - 2\varepsilon}}{-1 - \varepsilon + \sqrt{5 + \varepsilon^2 - 2\varepsilon}} \simeq 2.6$$

If one solves:

$$\begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 + \varepsilon \\ 2 \end{bmatrix}$$

Exact solution $x^* = (1, 1)$.

$$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \frac{1}{\varepsilon} & 1 \end{bmatrix} \times \begin{bmatrix} \varepsilon & 1 \\ 0 & 1 - \frac{1}{\varepsilon} \end{bmatrix}$$

ε	$\frac{\|x^* - x\|}{\|x^*\|}$	$\kappa_2(A)$
10^{-3}	6×10^{-16}	2.621
10^{-6}	2×10^{-11}	2.618
10^{-9}	9×10^{-8}	2.618
10^{-12}	9×10^{-5}	2.618
10^{-15}	7×10^{-2}	2.618

Table: Relative error as a function of ε.

- Even if A is well conditioned, Gaussian elimination may introduce errors

$$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \frac{1}{\varepsilon} & 1 \end{bmatrix} \times \begin{bmatrix} \varepsilon & 1 \\ 0 & 1 - \frac{1}{\varepsilon} \end{bmatrix}$$

ε	$\frac{\|x^* - x\|}{\|x^*\|}$	$\kappa_2(A)$
10^{-3}	6×10^{-16}	2.621
10^{-6}	2×10^{-11}	2.618
10^{-9}	9×10^{-8}	2.618
10^{-12}	9×10^{-5}	2.618
10^{-15}	7×10^{-2}	2.618

Table: Relative error as a function of ε.

- Even if A is well conditioned, Gaussian elimination may introduce errors
- Explanation: pivot ε is too small and leads to a large element growth (growth factor) in L and U: $\frac{1}{\varepsilon}$ in L leads to a loss of information/accuracy in $1 - \frac{1}{\varepsilon}$

$$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \frac{1}{\varepsilon} & 1 \end{bmatrix} \times \begin{bmatrix} \varepsilon & 1 \\ 0 & 1 - \frac{1}{\varepsilon} \end{bmatrix}$$

- Let us try to exchange rows 1 and 2 of A and b:

$$\begin{bmatrix} 1 & 1 \\ \varepsilon & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 + \varepsilon \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 \\ \varepsilon & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \varepsilon & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 - \varepsilon \end{bmatrix}$$

$\rightarrow$ Multipliers are bounded: $\forall i = k+1 : n, \quad \frac{|a_{i,k}^{(k)}|}{|a_{k,k}^{(k)}|} \leq 1$

$\rightarrow$ terms of original matrix remain significant in LU factors

$\rightarrow$ perfect accuracy obtained !

Partial Pivoting

- Partial pivoting: choose at each step the largest element of the column as the pivot

$\rightarrow$ avoids large elements in factors matrix (growth factor)

- Then (P: permutation), $PA = LU, \quad Ly = Pb, \quad Ux = y$
- LU with partial pivoting is practically *backward stable*

$$\frac{\|Ax - b\|}{\|A\| \times \|x\| + \|b\|} \approx \varepsilon \qquad (1)$$

$$\frac{\|x - x^*\|}{\|x^*\|} \approx \varepsilon \times \kappa(A) \qquad (2)$$

(1) small backward error (and small residual) independently of the conditioning

(2) accuracy depends on conditioning
if $\varepsilon \approx 10^{-q}$ et $\kappa(A) \approx 10^{p}$ then x has approximatively $(q - p)$ correct digits

LU factorization with partial pivoting

Next algorithm computes **L** and **U** such that **PA** = **LU**, and computes **Pb**.

Algorithm 4 LU factorization with partial pivoting

for $k = 1, n-1$ **do**
  Pivot search : Find index i of largest entry in $\mathbf{A}(k:n, k)$
  if $|A(i,k)| \leq \varepsilon \|A\|$ **then**
    exit since **A** is numerically singular
  end if
  Swap rows i and k of **A** and **b**
  A(k+1:n,k) = A(k+1:n,k) / A(k,k)
  A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - A(k+1:n,k)*A(k,k+1:n)
end for

To solve **Ax** = **b** one should thus perform **LU** = **Pb**. Permutation **P** is either applied on **b** during Algorithm 4 or needs be saved to be applied later.

Extensions to **LU** factorization with partial pivoting

Exercise 4 (LU with pivoting)

1. *Explain how to modify algorithm 4 to factor singular matrices in the form proposed at exercice 2 (so that using exercice 2 one could then also compute the null-space of* $\mathbf{A}$*)*
2. *Let us suppose then that in algorithm 4, we want at each step of* **Pivot search** *step to find the largest entry not only in the column (*$\mathbf{A}(k:n,k)$*) both also (*$\mathbf{A}(k:n,k:n)$*), so called* **total pivoting**. *Describe how algorithm 4 should be modified.*
3. *Compare algorithm proposed at questions 1 and 2.*

Linear Algebra Basics
Gaussian Elimination and LU factorization
LU Factorization with partial pivoting
Symmetric matrices
Cholesky Factorization

Symmetric matrices

- Assumption: A has a LU factorization
- A symmetric: only store lower or upper triangle
- $A = LU$ and $A^T = A \Rightarrow LU = U^T L^T$,
  thus $LU(L^T)^{-1} = U^T \Rightarrow (U)(L^T)^{-1} = L^{-1}U^T = D$ diagonal
  and $U = DL^T$,
  finally $A = L(DL^T) = LDL^T$, with $D = \text{Diag}(U)$ and
- Example:

$$\begin{bmatrix} 4 & -8 & -4 \\ -8 & 18 & 14 \\ -4 & 14 & 25 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 3 & 1 \end{bmatrix} \times \begin{bmatrix} 4 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix} \times \begin{bmatrix} 1 & -2 & -1 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix}$$

- Solution of $Ax = b$: with $A = LDL^T$:
  1. $Ly = b$ then $Dz = y$ followed by
  2. $L^T x = z$

Properties of LDL^T Algorithm

We have shown that if $\mathbf{A}$ symmetric and $A = LU$ exists then $\exists L$ and $D(= \text{Diag}(U))$ such that $A = LDL^T$. **LU** Algorithm 1 thus already computes all we need: L and D.

Proposition 1 (LDL^T Algorithm)

Given a symmetric matrix A for which an LU factorisation exists, the LU algorithm 1 can be adapted to compute LDL^T *factorization.*

Proposition 2 (Complexity of LDL^T factorization)

If only the lower triangular part of the matrix (including diagonal) is used/updated and if only L and D matrices are stored, then the cost of LDL^T *factorisation is* $\approx \frac{n^3}{3}$

LDL^T Algorithm for symmetric matrices

- let l_k be column k of L and u_k be row k of U then in Algorithm 1,
  $A(k+1:n, k+1:n) = A(k+1:n, k+1:n) - l_k * u_k^T$
- since $l_k = u_k / u_{kk}$, when U is not stored, one must temporarily save u_k to perform the update.

Algorithm 5 LDT^T factorization

for $k = 1, n-1$ **do**
  if $|\mathbf{A}(k,k)|$ too small exit (small pivots
  $\mathbf{v}_k$ = A(k+1:n,k) (corresponds to u_k in **LU** Agorithm)
  A(k+1:n,k) = A(k+1:n,k) / A(k,k)
  for $j = k+1, n$ **do**
    A(j:n,j) = A(j:n,j) - A(j:n,k)*$\mathbf{v}_k(j)$
  end for
end for

Symmetric matrices and pivoting

- Diagonal pivoting preserves symmetry but is insufficient for stability
- In general one looks for a permutation P such that:

$$PAP^T = LDL^T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ x & 1 & 0 & 0 \\ x & 0 & 1 & 0 \\ x & x & x & 1 \end{bmatrix} \times \begin{bmatrix} x & 0 & 0 & 0 \\ 0 & x & x & 0 \\ 0 & x & x & 0 \\ 0 & 0 & 0 & x \end{bmatrix} \times \begin{bmatrix} 1 & x & x & x \\ 0 & 1 & 0 & x \\ 0 & 0 & 1 & x \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- D: matrix of diagonal 1×1 and 2×2 blocks
  Examples of 2x2 pivots: $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, $\begin{bmatrix} \varepsilon_1 & 1 \\ 1 & \varepsilon_2 \end{bmatrix}$
- Pivot choice more complex: 2 columns at each step Let $PAP^T = \begin{bmatrix} E & C^T \\ C & B \end{bmatrix}$. If E is a 2x2 pivot, form E^{-1} to get:

$$PAP^T = \begin{bmatrix} I & 0 \\ CE^{-1} & I \end{bmatrix} \begin{bmatrix} E & 0 \\ 0 & B - CE^{-1}C^T \end{bmatrix} \begin{bmatrix} I & E^{-1}C^T \\ 0 & I \end{bmatrix}$$

Complexity of LDL^T factorization

for $k = 1, n-1$ **do**
  if $|\mathbf{A}(k,k)|$ too small exit (small pivots
  $\mathbf{v}_k$ = A(k+1:n,k) (corresponds to u_k in **LU** Agorithm)
  A(k+1:n,k) = A(k+1:n,k) / A(k,k)
  for $j = k+1, n$ **do**
    A(j:n,j) = A(j:n,j) - A(j:n,k)*$\mathbf{v}_k(j)$
  end for
end for

- $\text{flops}(LDL^T) = 2\sum_{k=1}^{n-1}\left(\sum_{i=1}^{n-k} i\right) = 2\sum_{k=1}^{n-1}\left(\frac{(n-k)(n-k+1)}{2}\right)$

$$\text{flops}(LDL^T) \approx \sum_{k=1}^{n-1}(n-k)^2 \qquad (\text{thus } \tfrac{1}{2}\text{flops}(\mathbf{LU}))$$

$$LDL^T \approx \frac{n^3}{3}\text{floating point operations}$$

Linear Algebra Basics
Gaussian Elimination and LU factorization
LU Factorization with partial pivoting
Symmetric matrices
Cholesky Factorization

- **A** positive definite if $\mathbf{x}^T\mathbf{A}\mathbf{x} > 0 \quad \forall \mathbf{x} \neq 0$
- **A** symmetric positive definite $\Rightarrow$ Cholesky factorization
  $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ with L lower triangular, **L** is unique
- By identification :

$$\begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{21} & a_{22} & a_{32} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \times \begin{bmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

- It follows:
$$\begin{array}{lll} a_{11} = l_{11}^2 & a_{21} = l_{21} \times l_{11} & a_{31} = l_{31} \times l_{11} \\ a_{22} = l_{21}^2 + l_{22}^2 & a_{32} = l_{31} \times l_{21} + l_{32} \times l_{22} & a_{33} = l_{31}^2 + l_{32}^2 + l_{33}^2 \end{array}$$

Thus:
$$\begin{array}{lll} l_{11} = \sqrt{a_{11}} & l_{21} = a_{21}/l_{11} & l_{31} = a_{31}/l_{11} \\ a_{22}^{(1)} = a_{22} - l_{21}^2 & a_{32}^{(1)} = a_{32} - l_{31} \times l_{21} & a_{33}^{(1)} = a_{33} - l_{31}^2 \\ l_{22} = \sqrt{a_{22}^{(1)}} & l_{32} = a_{32}^{(1)}/l_{22} & a_{33}^{(2)} = a_{33}^{(1)} - l_{32}^2 \\ l_{33} = \sqrt{a_{33}^{(2)}} & & \end{array}$$

Cholesky Factorization

```
do k=1, n
   A(k,k)=sqrt(A(k,k))
   A(k+1:n,k) = A(k+1:n,k)/A(k,k)
   do j=k+1, n
      A(j:n,j) = A(j:n,j) - A(j:n,k) A(j,k)
   end do
end do
```

- Cholesky is backward stable (without pivoting)
- Factorization: $\approx \frac{n^3}{3}$ flops
- Similar to LU, but only on the lower triangle. **LU** factorization:

```
A(k+1:n,k) = A(k+1:n,k)/A(k,k)
A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - &
                 A(k+1:n,k) * A(k,k+1:n)
```

Méthodes itératives

2022–2023

Introduction

On cherche une suite $x^{(p)}$ de $\mathbb{R}^n$ convergeant vers la solution x^* de $A \cdot x = b$

$$\forall x^{(0)} \in \mathbb{R}^n \qquad x^{(p+1)} = H(x^{(p)})$$

- La matrice A n'est jamais modifiée
- Problème du suivi de la convergence et du choix du test d'arrêt
- La solution obtenue n'est pas exacte
- La matrice doit vérifier des conditions de convergence
- La vitesse de convergence dépend de la valeur des coefficients de la matrice

Familles de méthodes

Algorithmes itératifs de relaxation

Algorithmes associés à une décomposition de A sous la forme $M - N$: Gauss-Seidel, Jacobi, SOR

Méthodes de gradient

Plus grande pente, directions conjuguées, gradient conjugué

Première partie I

Méthodes itératives de relaxation

Algorithme itératif de relaxation associé à une décomposition de A sous une forme $M - N$

Soit $A = M - N$ avec M inversible

$$\begin{aligned} A \cdot x^* = b &\iff (M-N)\cdot x^* = b \\ &\iff M \cdot x^* = N \cdot x^* + b \\ &\iff x^* = M^{-1} \cdot N \cdot x^* + M^{-1} \cdot b \end{aligned}$$

Algorithme itératif de relaxation associé :

$$\begin{cases} x^{(0)} \in \mathbb{R}^n \\ x^{(p+1)} = M^{-1} \cdot N \cdot x^{(p)} + M^{-1} \cdot b \\ \qquad\quad = M^{-1} \cdot (M - A) \cdot x^{(p)} + M^{-1} \cdot b \\ \qquad\quad = x^{(p)} + M^{-1} \cdot r^{(p)} \end{cases}$$

avec $r^{(p)} = b - A \cdot x^{(p)}$ le vecteur résidu

Coût d'une itération (en dehors du calcul du résidu) : résolution d'un système de matrice M

Introduction

Soit la décomposition de $A = D - E - F$ avec :

$$D = \begin{pmatrix} a_{11} & 0 & \dots & 0 \\ 0 & \ddots & & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \dots & 0 & a_{nn} \end{pmatrix} \qquad E = \begin{pmatrix} 0 & \dots & \dots & 0 \\ -a_{21} & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ -a_{n1} & \dots & -a_{nn-1} & 0 \end{pmatrix}$$

$$F = \begin{pmatrix} 0 & -a_{12} & \dots & -a_{1n} \\ \vdots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & -a_{n-1n} \\ 0 & \dots & \dots & 0 \end{pmatrix}$$

Deux variantes

Algorithme de Jacobi : $M = D$ et $N = E + F$

L'itération p consiste à résoudre l'équation :
$D \cdot x^{(p+1)} - (E + F) \cdot x^{(p)} = b$
C.N. d'application : D inversible $i = 1, \dots, n$ $a_{ii} \neq 0$

Algorithme de Gauss-Seidel : $M = D - E$ et $N = F$

L'itération p consiste à résoudre l'équation :
$(D - E) \cdot x^{(p+1)} - F \cdot x^{(p)} = b$
C.N. d'application : $(D - E)$ inversible $\iff$ $i = 1, \dots, n$ $a_{ii} \neq 0$

Condition d'arrêt

Mise en œuvre du test d'arrêt (à la fin de chaque itération) Choix d'une norme vectorielle $|\cdot|$, d'une précision ε puis d'un test d'arrêt :

- $\left\|x^{(p+1)} - x^{(p)}\right\| < \varepsilon$: simple mais numériquement dangereux (risque d'arrêt prématuré loin de la solution).
- $\left\|b - A \cdot x^{(p+1)}\right\| / \|b\| < \varepsilon$: numériquement plus sûr.

La mise en œuvre de ce test d'arrêt n'entraîne pas de calculs supplémentaires

Pour Jacobi : $x^{(p+1)} = D^{-1} \cdot \left[b - A.x^{(p)}\right] + x^{(p)}$
Pour Gauss-Seidel : $x^{(p+1)} = (D - E)^{-1} \cdot \left[b - A.x^{(p)}\right] + x^{(p)}$

Convergence d'un algorithme issu d'une décomposition de $A = M - N$

Soit $x^{(p)} = x^* + \varepsilon^{(p)}$

$$\left.\begin{array}{r} x^{(p+1)} = M^{-1} \cdot N \cdot x^{(p)} + M^{-1} \cdot b \\ x^* = M^{-1} \cdot N \cdot x^* + M^{-1} \cdot b \end{array}\right\} \implies \varepsilon^{(p+1)} = M^{-1} \cdot N \cdot \varepsilon^{(p)}$$

$$\implies \varepsilon^{(p)} = \left(M^{-1} \cdot N\right)^p \cdot \varepsilon^{(0)}$$

$$\begin{aligned} \text{Convergence} &\iff \forall x^{(0)} \in \mathbb{R}^n \lim_{p \to +\infty} x^{(p)} = x^* \\ &\iff \forall \varepsilon^{(0)} \in \mathbb{R}^n \lim_{p \to +\infty} \varepsilon^{(p)} = 0 \\ &\iff \forall \varepsilon^{(0)} \in \mathbb{R}^n \lim_{p \to +\infty} \left(M^{-1} \cdot N\right)^p \cdot \varepsilon(0) = 0 \\ &\iff \lim_{p \to +\infty} \left(M^{-1} \cdot N\right)^p = 0 \\ &\iff \rho\left(M^{-1} \cdot N\right) < 1 \quad \text{(propriété admise)} \end{aligned}$$

CNS de convergence

CNS de convergence (admise)

$$\lim_{p \to +\infty} \left(M^{-1} \cdot N\right)^p = 0 \iff \rho\left(M^{-1} \cdot N\right) < 1$$

ρ désignant le rayon spectral d'une matrice : soient $\lambda_i \in \mathbb{C}, i = 1, \ldots, n$ les valeurs propres d'une matrice A :

$$\rho(A) = \max_{i=1,\ldots,n} |\lambda_i|$$

Autre version de la CNS de convergence :

$$\forall \lambda \in \mathbb{C} \text{ valeur propre de } M^{-1} \cdot N \ : \ |\lambda| < 1$$

Première possibilité pour vérifier la convergence : utiliser la CNS
Jacobi : $\rho(D^{-1} \cdot (E + F)) < 1$
Gauss-Seidel : $\rho((D - E)^{-1} \cdot F) < 1$

Exercice 1

Soit le système $A \cdot x = b$ avec $A \in \mathcal{M}_n(\mathbb{R})$ inversible.
On considère une décomposition de A sous la forme $M - N$.
On supposera (pour simplifier) que toutes les valeurs propres de la matrice $M^{-1} \cdot N$ sont réelles et que tous les vecteurs propres associés appartiennent à $\mathbb{R}^n$.
Montrer la propriété suivante (réciproque de la propriété admise en cours) :

La méthode itérative de relaxation associée à la décomposition de A converge vers x quelque soit le vecteur initial $\implies \rho(M^{-1} \cdot N) < 1$

CS de convergence

Deuxième possibilité pour vérifier la convergence : utiliser des CS spécifiques (déduites de la CNS)

A à diagonale dominante (stricte) : $i = 1, \ldots, n \qquad |a_{ii}| \underset{>}{\geqslant} \sum_{j=1, j \neq i}^{n} |a_{ij}|$

Théorème 1 : Si A est à diagonale dominante stricte, alors les algorithmes de Jacobi et de Gauss-Seidel convergent quelque soit le vecteur de départ.
Théorème 2 : Si A est à diagonale dominante et si :

$$\exists k \in \{1, \ldots, n\} \ |a_{kk}| > \sum_{j=1, j \neq k}^{n} |a_{kj}|$$

alors l'algorithme de Gauss-Seidel converge quelque soit le vecteur de départ.
Théorème 3 : Si A est symétrique définie positive, alors l'algorithme de Gauss-Seidel converge quelque soit le vecteur de départ.

Méthode SOR (successive over-relaxation)

Technique d'accélération de Gauss-Seidel

$$x_i^{(p+1)} = \omega(\text{Gauss-Seidel}) + (1-\omega)x_i^{(p)}$$

$$x_i^{(p+1)} = \frac{\omega}{a_{ii}}(b_i - \sum_{j>i} a_{ij}x_j^{(p)} - \sum_{j<i} a_{ij}x_j^{(p+1)}) + (1-\omega)x_i^{(p)}$$

C'est un algorithme itératif de relaxation issu d'une décomposition de A sous la forme $M_\omega - N_\omega$ avec $M_\omega = \frac{D}{\omega} - E$ et $N_\omega = \left(\frac{1}{\omega} - 1\right) D + F$

Conditions sur ω pour assurer la convergence ?

Quelques résultats pour des cas particuliers :

- A symétrique définie positive et tridiagonale par blocs, un seul paramètre optimal ω : $\omega = \frac{2}{1+\sqrt{1-\rho(D^{-1}\cdot(E+F))^2}}$

Deuxième partie II

Méthodes de gradient

Principes

Rappel : si $F : \mathbb{R}^n \to \mathbb{R}$ et $x \in \mathbb{R}^n$, $grad(F)(x) = \begin{pmatrix} \frac{\partial F}{\partial x_1}(x) \\ \vdots \\ \frac{\partial F}{\partial x_n}(x) \end{pmatrix}$

Propriété 1

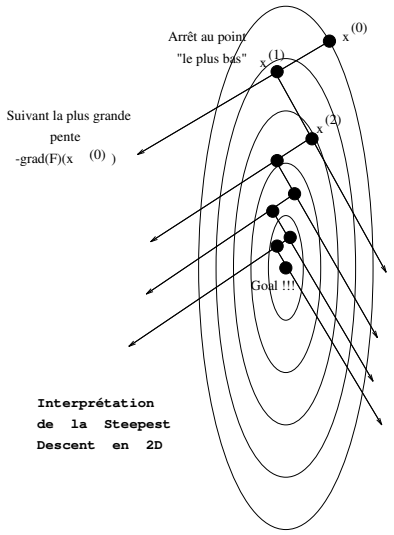
$$F : \mathbb{R}^n \to \mathbb{R}$$
$$x \mapsto F(x) = \frac{1}{2}x^T \cdot A \cdot x - x^T \cdot b$$

Si A symétrique, $A \cdot x - b = grad(F)(x)$ ou $b - A \cdot x = -grad(F)(x)$

Propriété 2

Si A symétrique définie positive (supposé vrai pour toute méthode de gradient) : $A \cdot x^* = b \iff \forall x' \neq x^* \qquad F(x') > F(x^*)$

Interprétation de la "steepest descent" en 2D



Méthode de la Steepest Descent

A symétrique définie positive, $x^{(0)} \in \mathbb{R}^n$ quelconque
Au cours de l'itération p, $x^{(p+1)}$ se déduire de $x^{(p)}$ par :

$$x^{(p+1)} = x^{(p)} + \lambda_p r^{(p)}$$

- $r^{(p)}$: direction de descente (vecteur)
- λ_p : progression dans la direction de descente (scalaire)

Choix des paramètres de la descente :

- Direction : suivant la plus grande pente i.e. direction opposée au gradient en $x^{(p)}$: $r^{(p)} = -grad(F)(x^{(p)}) = b - A \cdot x^{(p)}$
- Progression : recherche du point "le plus bas" dans la direction $r^{(p)}$ obtenu en minimisant :

$$f(\lambda) = F(x^{(p)} + \lambda r^{(p)}) - F(x^{(p)}) = \frac{1}{2}\lambda^2 (r^{(p)})^T \cdot A \cdot r^{(p)} - \lambda (r^{(p)})^T \cdot r^{(p)}$$

Le minimum de cette fonction de λ_p est atteint en $\lambda_p = \frac{(r^{(p)})^T \cdot r^{(p)}}{(r^{(p)})^T \cdot A \cdot r^{(p)}}$

Algorithme

D'où l'algorithme :
$x^{(0)} = \dots$
$r^{(0)} = b - A \cdot x^{(0)}$
$p = 1$
Tant que $\frac{\|r^{(p-1)}\|}{\|b\|} > \varepsilon$ faire

$$\lambda_{p-1} = \frac{(r^{(p-1)})^T \cdot r^{(p-1)}}{(r^{(p-1)})^T \cdot A \cdot r^{(p-1)}}$$
$$x^{(p)} = x^{(p-1)} + \lambda_{p-1} r^{(p-1)}$$
$$r^{(p)} = b - A \cdot x^{(p)}$$
$$p = p + 1$$

Coût d'une itération : 2 produits matrice-vecteur mais
$r^{(p)} = b - A \cdot x^{(p)} = b - A \cdot x^{(p-1)} - \lambda_{p-1} A \cdot r^{(p-1)}$
$r^{(p)} = r^{(p-1)} - \lambda_{p-1} A \cdot r^{(p-1)}$

Propriétés de la Steepest Descent

- Deux directions de descente successives $r^{(p+1)}$ et $r^{(p)}$ sont orthogonales.
- Soit u vecteur propre de A. Si $x^{(0)}$ vérifie :
  $\exists \beta \in \mathbb{R}, \beta \neq 0$ t.q. $x^* - x^{(0)} = \beta u$, la SD atteint la solution exacte en une itération.
- Si toutes les valeurs propres de la matrice A sont égales (une seule valeur propre), alors, $\forall x^{(0)}$, la SD atteint la solution exacte en une itération.

Propriétés de la Steepest Descent

- Soit u vecteur propre de A. Si $x^{(0)}$ vérifie :
  $\exists \beta \in \mathbb{R}, \beta \neq 0$ t.q. $x^* - x^{(0)} = \beta u = \varepsilon^{(0)}$, la SD atteint la solution exacte en une itération.

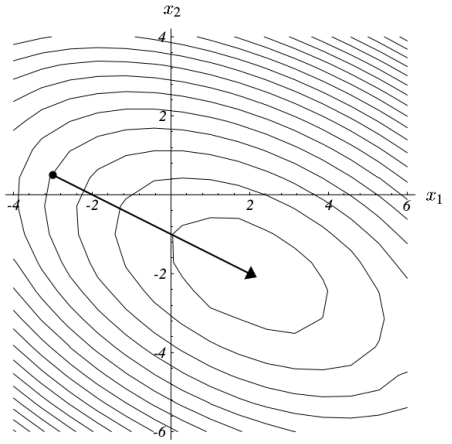


Figure 14: Steepest Descent converges to the exact solution on the first iteration if the error term is an eigenvector.

Propriétés de la Steepest Descent

- Si toutes les valeurs propres de la matrice A sont égales (une seule valeur propre), alors, $\forall x^{(0)}$, la SD atteint la solution exacte en une itération.

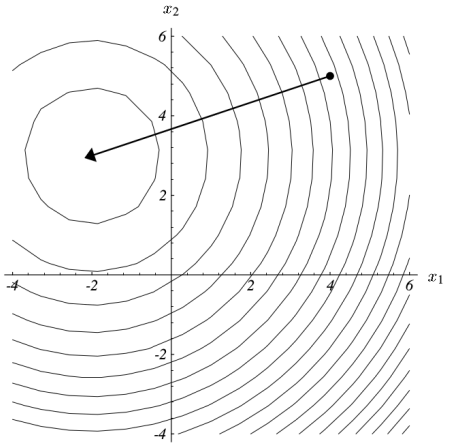
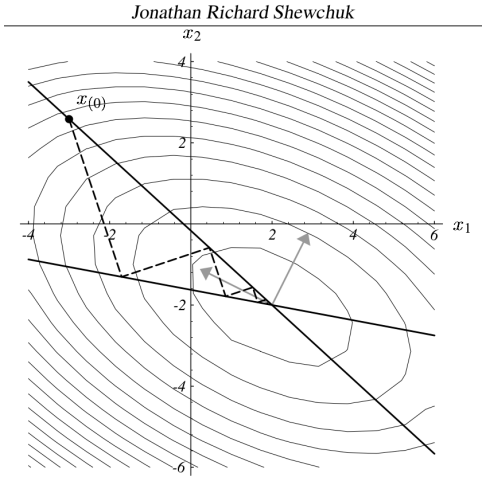


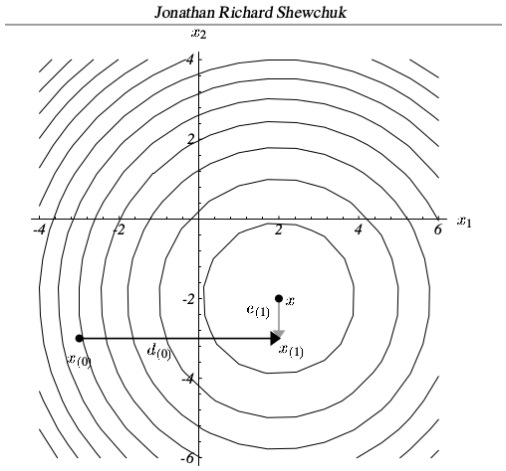
Figure 15: Steepest Descent converges to the exact solution on the first iteration if the eigenvalues are all equal.

cas défavorable de la "steepest descent" en 2D



Référence : *An Introduction to the Conjugate Gradient Method without the Agonizing Pain*, **Jonathan Richard Shewchuk**

Idée : ne considérer une direction qu'une fois



Problème : pour savoir où s'arrêter dans une direction, il faut connaître la suivante

Solution : considérer des directions A-conjuguées

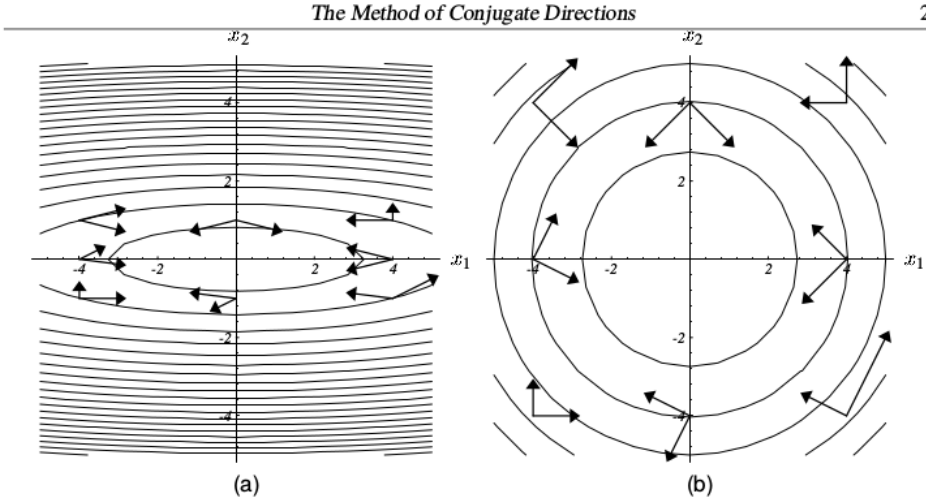


Figure 22: These pairs of vectors are A-orthogonal . . . because these pairs of vectors are orthogonal.

A-orthogonalisation de Gram-Schmidt

Soit u_i $i = 0, \ldots, n-1$ une base quelconque de $\mathbb{R}^n$ et $A \in \mathcal{M}_n(\mathbb{R})$ symétrique définie positive.
On considère le produit scalaire suivant (A-produit scalaire) :

$$\forall (x, y) \in (\mathbb{R}^n)^2, (x|y)_A = (x|A \cdot y)$$

On veut construire une nouvelle base d_i $i = 0, \ldots, n-1$ qui soit A-orthogonale i.e. $\forall i \neq j, (d_i|d_j)_A = 0$

$$\begin{aligned} d_0 &= u_0 \\ d_1 &= u_1 + \beta_{10} d_0 \\ d_2 &= u_2 + \beta_{20} d_0 + \beta_{21} d_1 \\ d_{p+1} &= u_{p+1} + \sum_{j=0}^{p} \beta_{p+1j} d_j \\ &\ldots \\ d_{n-1} &= u_{n-1} + \sum_{j=0}^{n-2} \beta_{nj} d_j \end{aligned}$$

A-orthogonalisation de Gram-Schmidt – suite

$$d_{p+1} = u_{p+1} + \sum_{j=0}^{p} \beta_{p+1j} d_j$$

Calcul de d_{p+1} en supposant $d_i, i = 0, \ldots, p$ déjà calculés et $\forall j \neq i, (d_j|d_i)_A = 0$:

$$\begin{aligned} i = 0, \ldots, p \qquad (d_{p+1}|d_i)_A &= (u_{p+1}|d_i)_A + \sum_{j=0}^{p} \beta_{p+1j} (d_j|d_i)_A \\ 0 &= (u_{p+1}|d_i)_A + \beta_{p+1i} (d_i|d_i)_A \end{aligned}$$

D'où

$$\beta_{p+1i} = -\frac{(u_{p+1}|d_i)_A}{(d_i|d_i)_A} = -\frac{(u_{p+1}|A \cdot d_i)}{(d_i|A \cdot d_i)}$$

Exercice 2 (avec nouvelles notations)

Soit à résoudre $A \cdot x = b$ avec $A \in \mathcal{M}_n(\mathbb{R})$ symétrique définie positive.
On note $u_i,\ i = 0, \ldots, n-1$ les différentes colonnes de A.
Le procédé de A-orthogonalisation de Gram-Schmidt est appliqué à $u_i,\ i = 0, \ldots, n-1$ pour construire une base $d_i,\ i = 0, \ldots, n-1$ A-orthogonale de $\mathbb{R}^n$.

1. Exprimer la solution du système $A \cdot x^* = b$ dans la base $d_i,\ i = 0, \ldots, n-1$.
2. Montrer que $\forall y \in \mathbb{R}^n, x^* = y + \sum_{i=0}^{n-1} \frac{(d_i|b - A \cdot y)}{(d_i|A \cdot d_i)} d_i$
3. On considère l'algorithme itératif suivant :

$$\forall x^{(0)} \in \mathbb{R}^n,\ x^{(p+1)} = x^{(p)} + \alpha_p d_p \text{ avec } \alpha_p = \frac{(d_p|b - A \cdot x^{(p)})}{(d_p|A \cdot d_p)}$$

   3.1 Donner la relation entre $x^{(p)}$ et $x^{(0)}$
   3.2 Montrer que pour $p = 1, 2, 3, \ldots (d_p|A \cdot x^{(p)}) = (d_p|A.x^{(0)})$
4. Montrer que l'algorithme de la question ci-dessus atteint exactement la solution du système $A \cdot x^* = b$ en n itérations.

Quelques propriétés

Propriété 0 (question 3 de l'exercice)

$p = 1, 2, 3, \ldots,\ (d_p|A \cdot x^{(p)}) = (d_p|A \cdot x^{(0)}) \implies (d_p|r^{(p)}) = (d_p|r^{(0)})$

Propriété 1

$j = 0, \ldots, p-1,\ (r^{(p)}|d_j) = 0$

Propriété 2

$j = 0, \ldots, p-1,\ (r^{(p)}|u_j) = 0$

Propriété 3

$(r^{(p)}|d_p) = (r^{(p)}|u_p)$

Mise en forme d'une itération de l'algorithme :

$x^{(0)} = ?,\ r^{(0)} = b - Ax^{(0)}$
$\{u_i\}_{i=0,\dots,n-1}$ colonnes de A
$d_0 = u_0$
Pour $p = 0, n-1$

$\alpha_p = \frac{(d_p|r^{(p)})}{(d_p|A\cdot d_p)}$
$x^{(p+1)} = x^{(p)} + \alpha_p d_p$
$r^{(p+1)} = r^{(p)} - \alpha_p A\cdot d_p$
pour $j = 0,\dots,p,\ \beta_{p+1j} = -\frac{(u_{p+1}|A\cdot d_j)}{(d_j|A\cdot d_j)}$
$d_{p+1} = u_{p+1} + \sum_{j=0}^{p} \beta_{p+1j} d_j$

Fin Pour

Coût : mémoire, calcul ?

Cas particulier du Gradient Conjugué

Soit $u_i = r^{(i)},\ i = 0,\dots,n-1$

- 1ère conséquence :

$$\alpha_p = \frac{(d_p|r^{(p)})}{(d_p|A\cdot d_p)} = \frac{(r^{(p)}|r^{(p)})}{(d_p|A\cdot d_p)}$$

- 2ème conséquence : $j = 0,\dots,p$

$$\beta_{p+1j} = -\frac{(u_{p+1}|A\cdot d_j)}{(d_j|A\cdot d_j)} = -\frac{(r^{(p+1)}|A\cdot d_j)}{(d_j|A\cdot d_j)}$$

D'autre part $r^{(j+1)} = r^{(j)} - \alpha_j A\cdot d_j$

$$(r^{(p+1)}|r^{(j+1)}) = (r^{(p+1)}|r^{(j)}) - \alpha_j (r^{(p+1)}|A\cdot d_j)$$

et donc

$$\alpha_j (r^{(p+1)}|A\cdot d_j) = (r^{(p+1)}|r^{(j)}) - (r^{(p+1)}|r^{(j+1)})$$

Cas particulier du gradient conjugué – suite

$$\alpha_j (r^{(p+1)}|A\cdot d_j) = (r^{(p+1)}|r^{(j)}) - (r^{(p+1)}|r^{(j+1)})$$

$j = p$

$$\begin{aligned}
\alpha_p (r^{(p+1)}|A\cdot d_p) &= (r^{(p+1)}|r^{(p)}) - (r^{(p+1)}|r^{(p+1)}) \\
&= 0 - (r^{(p+1)}|r^{(p+1)}) \quad \text{(prop. 2)} \\
(r^{(p+1)}|A\cdot d_p) &= -\frac{1}{\alpha_p}(r^{(p+1)}|r^{(p+1)}) \\
(r^{(p+1)}|A\cdot d_p) &= -\frac{(d_p|A\cdot d_p)}{(r^p|r^{(p)})}(r^{(p+1)}|r^{(p+1)}) \\
-\frac{(r^{(p+1)}|A\cdot d_p)}{(d_p|A.d_p)} &= \frac{(r^{(p+1)}|r^{(p+1)})}{(r^p|r^{(p)})} \\
\Longrightarrow\ \beta_{p+1p} &= \frac{(r^{(p+1)}|r^{(p+1)})}{(r^p|r^{(p)})}
\end{aligned}$$

$j = 0,\dots,p-1$

$$\begin{aligned}
\alpha_j (r^{(p+1)}|A\cdot d_j) &= (r^{(p+1)}|r^{(j)}) - (r^{(p+1)}|r^{(j+1)}) \\
&= 0 - 0 \quad \text{(prop. 2)} \\
\Longrightarrow\ \beta_{p+1j} &= \frac{(r^{(p+1)}|A\cdot d_j)}{(d_j|A\cdot d_j)} = 0
\end{aligned}$$

d'où l'expression de d_{p+1} :

$$d_{p+1} = r^{(p+1)} + \beta_{p+1p} d_p \quad \text{avec} \quad \beta_{p+1p} = \frac{(r^{(p+1)}|r^{(p+1)})}{(r^{(p)}|r^{(p)})}$$

Algorithme final du Gradient Conjugué

$x^{(0)} = ?,\ d^{(0)} = b - Ax^{(0)},\ r^{(0)} = d^{(0)},\ p = 0$
Boucler

$\alpha_p = \frac{(r^{(p)}|r^{(p)})}{(d_p|A\cdot d_p)}$
$x^{(p+1)} = x^{(p)} + \alpha_p d_p$
$r^{(p+1)} = r^{(p)} - \alpha_p A\cdot d_p$
$\beta_p = \beta_{p+1p} = \frac{(r^{(p+1)}|r^{(p+1)})}{(r^{(p)}|r^{(p)})}$
$d_{p+1} = r^{(p+1)} + \beta_p d_p$
$p = p + 1$

Jusqu'à Convergence

Coût : mémoire, calcul ?
Convergence ?

Dense linear algebra : QR factorization

P. Amestoy, P. Berger, M. Daydé, F.-H. Rouet
(INPT-ENSEEIHT)
and
J.-Y. L'Excellent (INRIA/LIP-ENS Lyon)

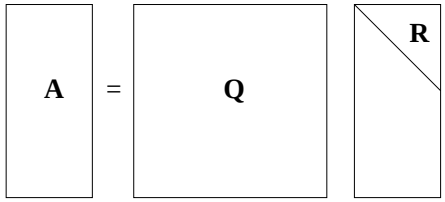
2020-2021

Outline

QR Factorization

QR Factorization

- Definition of an orthonormal set $\{x_1, \ldots, x_k\}$
  - $x_i^T x_j = 0 \quad \forall i \neq j$
  - $x_i^T x_i = 1$
- Orthogonal matrix Q: columns of Q form an orthonormal set $Q^T Q = I$, $Q^{-1} = Q^T$
- *QR* Factorization:



- Q: orthogonal.
- R: upper trapezoidal.

Example

$$\begin{bmatrix} 1 & -8 \\ 2 & -1 \\ 2 & 14 \end{bmatrix} = \begin{bmatrix} \frac{1}{3} & -\frac{2}{3} & -\frac{2}{3} \\ \frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & \frac{2}{3} & -\frac{1}{3} \end{bmatrix} \begin{bmatrix} 3 & 6 \\ 0 & 15 \\ 0 & 0 \end{bmatrix} = Q \times R$$

QR Factorization

- Factorization built by applying a succession of orthogonal transformations to the data:

$$Q = Q_1 \dots Q_n$$

with Q_i orthogonal matrices such that $Q^T A = R$

- Transformations:
  - Householder reflections.
  - Givens rotations.
  - Gram-Schmidt process (in this case $Q \in \mathcal{M}_{m,n}(\mathbb{K})$ and $R \in \mathcal{M}_n(\mathbb{K})$)

QR Factorization - Projections

Exercice 1 (Orthogonal projection)

Let $Q \in \mathcal{M}_{m,n}(\mathbb{R})$, $m \geq n$ be such that $Q^T.Q = I_n \in \mathcal{M}_n$. Let X be the n-dimensional span of the orthogonal set made of the columns of Q.
Show that $P_X = Q.Q^T \in \mathcal{M}_m(\mathbb{R})$ is the orthogonal projection onto X.

QR Factorization - Preamble

Definition 1 (Householder matrix)

$\forall \mathbf{u} \in I\!R^n, \mathbf{u} \neq 0$, *the* **Householder** *matrix* $\mathbf{H}_u \in M_{n,n}(I\!R)$ *is defined such that:*
$\mathbf{H}_u = \mathbf{I} - \frac{2}{\mathbf{u}^{\mathrm{T}}\mathbf{u}}\mathbf{u}\mathbf{u}^{\mathrm{T}}$.

Proposition 1

$\mathbf{H}_u$ *is orthogonal (* $\mathbf{H}_u^{\mathrm{T}}\mathbf{H}_u = \mathbf{I}$*) and symmetric.*

Proof.

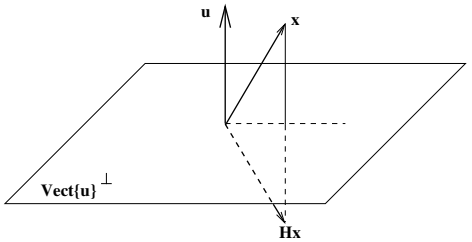
$\mathbf{H}_u$ is obviously symmetric: $\mathbf{H}_u^{\mathrm{T}} = \mathbf{H}_u$.
One has $\mathbf{H}_u^{\mathrm{T}}\mathbf{H}_u = \left(\mathbf{I} - \frac{2}{\mathbf{u}^{\mathrm{T}}\mathbf{u}}\mathbf{u}\mathbf{u}^{\mathrm{T}}\right)^2 = \mathbf{I} + \frac{4}{\left(\mathbf{u}^{\mathrm{T}}\mathbf{u}\right)^2}\mathbf{u}\mathbf{u}^{\mathrm{T}}\mathbf{u}\mathbf{u}^{\mathrm{T}} - \frac{4}{\mathbf{u}^{\mathrm{T}}\mathbf{u}}\mathbf{u}\mathbf{u}^{\mathrm{T}}$.
Therefore, $\mathbf{H}_u^{\mathrm{T}}\mathbf{H}_u = \mathbf{I} + \frac{4}{\mathbf{u}^{\mathrm{T}}\mathbf{u}}\mathbf{u}\mathbf{u}^{\mathrm{T}} - \frac{4}{\mathbf{u}^{\mathrm{T}}\mathbf{u}}\mathbf{u}\mathbf{u}^{\mathrm{T}} = \mathbf{I}$. □

Interpretation: $\mathbf{H}_u = \mathbf{I} - \frac{2}{\mathbf{u}^{\mathrm{T}}\mathbf{u}}\mathbf{u}\mathbf{u}^{\mathrm{T}}$.

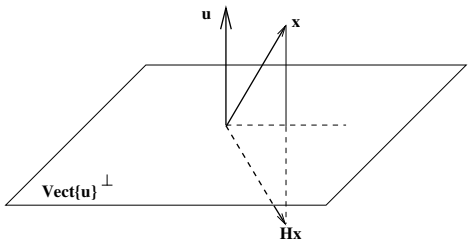
Proposition 2

- $\mathbf{H}_u\mathbf{u} = -\mathbf{u}$
- $\forall \mathbf{x} \in I\!R^n$, $\mathbf{H}_u\mathbf{x} = \mathbf{x}$ *if* $\mathbf{x}$ *and* $\mathbf{u}$ *orthogonal.*

Geometric interpetation:
$\forall \mathbf{x} \neq 0$, $\mathbf{H}_u\mathbf{x}$ is the symmetric of $\mathbf{x}$ with respect to $\mathrm{span}(\mathbf{u})^{\perp}$
Therefore, $\mathbf{H}_u$ is a reflection.



Another view on: $\mathbf{H}_u = \mathbf{I} - \frac{2}{\mathbf{u}^T\mathbf{u}}\mathbf{u}\mathbf{u}^T$.



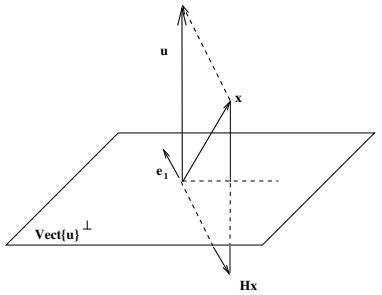
Let $\mathbf{Q}_u$ be such that $\mathbf{Q}_u = \frac{\mathbf{u}}{\|u\|_2}$.

From Exercice 1, $\mathbf{Q}_u\mathbf{Q}_u^T$ is a projection onto span($\mathbf{u}$): $\frac{\mathbf{u}\mathbf{u}^T}{\mathbf{u}^T\mathbf{u}}$ is the orthogonal projection onto span($\mathbf{u}$).
Therefore,
$\mathbf{H}_u = \mathbf{I} - 2\mathbf{Q}_u\mathbf{Q}_u^T$ is a symmetry with respect to span($\mathbf{u}$)$^\perp$.

$\mathbf{H}_u = \mathbf{I} - \frac{2}{\mathbf{u}^T\mathbf{u}}\mathbf{u}\mathbf{u}^T$: towards the QR factorization?

$\forall \mathbf{x} \in \mathbb{R}^n$ ($\mathbf{x} \neq 0$), can we find $\mathbf{u}$ such that $\mathbf{H}_u\mathbf{x}$ and e_1 collinear?

From a given $\mathbf{x}$, $\mathbf{u}$ must be easy to compute.



Defining $\mathbf{u} = \mathbf{x} + \sigma\ \mathbf{e}_1$, we look for σ such that:

$$\mathbf{H}_u\mathbf{x} = \begin{bmatrix} -\sigma \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Computation of σ / $\mathbf{H}_u\mathbf{x} = -\sigma\mathbf{e}_1$

Proposition 3

Let $\mathbf{x} \in \mathbb{R}^n$ be such that $\mathbf{x} \neq 0$, $x = (x_i)_{i=1,n}$ and $\sigma = sign(x_1)\|\mathbf{x}\|$.
Defining $\mathbf{u} = \mathbf{x} + \sigma\ \mathbf{e}_1$, then
$\mathbf{H}_u\mathbf{x} = -\sigma\mathbf{e}_1$, with e_1 the first vector of the standard basis.

Proof.

With $\mathbf{u} = \mathbf{x} + \sigma\ \mathbf{e}_1$ ($u \neq 0$) we have
$\mathbf{H}_u\mathbf{x} = \mathbf{H}_u\mathbf{u} - \sigma\mathbf{H}_u\mathbf{e}_1 = -\mathbf{u} - \sigma\left(\mathbf{e}_1 - \frac{2}{\mathbf{u}^T\mathbf{u}}\mathbf{u}\mathbf{u}^T\mathbf{e}_1\right) = -\sigma\mathbf{e}_1 + \frac{2\sigma u_1}{\mathbf{u}^T\mathbf{u}}\mathbf{u} - \mathbf{u}$.
Furthermore,
$\mathbf{u}^T\mathbf{u} = \mathbf{x}^T\mathbf{u} + \sigma\mathbf{e}_1^T\mathbf{u} = \mathbf{x}^T(\mathbf{x} + \sigma\ \mathbf{e}_1) + \sigma(\sigma + x_1) = \mathbf{x}^T\mathbf{x} + 2\sigma x_1 + \sigma^2$
Since $\mathbf{x}^T\mathbf{x} = \sigma^2$ and $u_1 = x_1 + \sigma$,

$$\mathbf{u}^T\mathbf{u} = 2\sigma(x_1 + \sigma) = 2\sigma u_1.$$

Therefore $\mathbf{H}_u\mathbf{x} = -\sigma\mathbf{e}_1$. □

The choice of σ in Proposition 3 is motivated by $\mathbf{u}^T\mathbf{u} \sim 0$ if $\mathbf{x} \sim \mathbf{e}_1$

Householder reflections: an example

$\mathbf{H} = I - 2\mathbf{v}.\mathbf{v}^T$ with $\mathbf{v} \in \mathbb{R}^n$ such that $\|\mathbf{v}\|_2 = 1$.
$\mathbf{H}$ is orthogonal and symmetric.
It provides a way of vanishing all the entries of a vector except for one component.

- Example :

$$x = \begin{bmatrix} 2 \\ -1 \\ 2 \end{bmatrix} \quad u = x + \begin{bmatrix} \|x\|_2 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 \\ -1 \\ 2 \end{bmatrix} \text{ and } \mathbf{v} = \frac{u}{\|u\|_2}$$

Then,

$$\mathbf{H}_u = I - 2\mathbf{v} \times \mathbf{v}^T = \frac{1}{15} \times \begin{bmatrix} -10 & 5 & -10 \\ 5 & 14 & 2 \\ -10 & 2 & 11 \end{bmatrix}$$

Therefore,

$$\mathbf{H}_u \times x = \begin{bmatrix} -3 \\ 0 \\ 0 \end{bmatrix}$$

QR Factorization with Householder reflections: existence

Theorem 1

Let $\mathbf{A} \in M_n(\mathbb{R})$ be a full rank matrix.
Then, there are $\mathbf{Q} \in M_n(\mathbb{R})$ orthogonal and $\mathbf{R} \in M_n(\mathbb{R})$ upper triangular such that

$$\mathbf{A} = \mathbf{QR}$$

Proof.

see Exercice 2. □

What about rectangular matrices?

The previous theorem can easily be extended to any matrix $\mathbf{A} \in M_{m,n}(\mathbb{R})$ $m \geq n$, with a rank equal to n. It can be shown that $\exists \mathbf{R} \in \mathbf{M}_n$ upper triangular such that

$$\mathbf{A} = \mathbf{Q} \begin{pmatrix} \mathbf{R} \\ 0 \end{pmatrix}$$

Householder Triangularization

Exercice 2

Let $\mathbf{A} \in M_n(\mathbb{R})$ be a full rank matrix.

1. *Show there exists $\mathbf{H}_1 \in M_n(\mathbb{R})$ orthogonal and $\alpha_1 \in \mathbb{R}$ such that $\mathbf{H}_1\mathbf{A}(\mathbf{e}_1) = \alpha_1\mathbf{e}_1$*
2. *Deduce from 1. that there exists a sequence of orthogonal matrices $\mathbf{H}_i, 1 \leq i \leq n-1$ such that $\mathbf{H}_{n-1} \dots \mathbf{H}_1\mathbf{A} = \mathbf{R}$ with $\mathbf{R} \in M_n(\mathbb{R})$ triangular.*

Definition 2

With $\mathbf{Q}^T = \mathbf{H}_{n-1} \dots \mathbf{H}_1$, exercice 2 leads to the Householder factorization $\mathbf{A} = \mathbf{QR}$

Householder Factorization: example

From a Householder vector: $\mathbf{u} = \mathbf{x} \pm \|x\|_2 e_1$, and its normalization $\mathbf{v} = \mathbf{u}/\|\mathbf{u}\|_2$ we can obtain a matrix that reads:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & a_{32} & a_{33} \\ 0 & a_{42} & a_{43} \\ 0 & a_{52} & a_{53} \end{bmatrix}$$

Let $\mathbf{H}$ be such that:

$$\mathbf{H} \times \begin{bmatrix} a_{22} \\ a_{32} \\ a_{42} \\ a_{52} \end{bmatrix} = \begin{bmatrix} a'_{22} \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{If } \mathbf{H}' = \begin{bmatrix} 1 & 0 \\ 0 & \mathbf{H} \end{bmatrix} \text{ then } \mathbf{H}' \times A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a'_{22} & a'_{23} \\ 0 & 0 & a'_{33} \\ 0 & 0 & a'_{43} \\ 0 & 0 & a'_{53} \end{bmatrix}$$

- Triangularization of a matrix 4×3 : $Q = \mathbf{H}_3 \times \mathbf{H}_2 \times H_1$

```
| x  x  x |     | x  x  x |     | x  x  x |
| x  x  x | H1  | 0  x  x | H2  | 0  x  x |
| x  x  x | ->  | 0  x  x | ->  | 0  0  x |
| x  x  x |     | 0  x  x |     | 0  0  x |
                                 | 0  0  0 |

      | x  x  x |
   H3 | 0  x  x |
   -> | 0  0  x | = R
      | 0  0  0 |
```

- QR backward stable, with a lower backward error than LU.

Computational complexity of the **QR** Factorization

Exercice 3 (Memory consumption)

Compute the memory space required to do a QR factorization.

Exercice 4 (Computational complexity)

Estimate the amount of operations that are done during the Householder factorization.

Remark: the most expensive part of the algorithm occurs, at each step, when updating the reduce matrix $\mathbf{A}_k \in M_{n-k,n-k}(\mathbb{R})$.

$$\mathbf{A}_k = \mathbf{A}_k - \beta \mathbf{u}_k (\mathbf{u}_k^{\mathrm{T}} \mathbf{A}_k) \quad avec \ \beta = \frac{2}{\mathbf{u}_k^{\mathrm{T}} \mathbf{u}_k} \quad et \ \mathbf{u}_k \in \mathbb{R}^{n-k} . \qquad (1)$$

Correction

Memory consumption

We do not compute $\mathbf{H}_i$: only the vectors $\mathbf{u}_i$ are required.
The triangular part of $\mathbf{A}$ is used to store these vectors (plus a vector of size n for the diagonal terms).
Therefore, the factorization can be done using the memory space allocated for the initial matrix (and a vector of size n).

Remark 1: **Q**-matrix product

Matrix product with $\mathbf{Q}$: sequence of Householder transformations.

Remark 2: Computational complexity

We need to specify the **QR** algorithm.

Householder **QR** factorization algorithm

Function: $[\mathbf{v}, \beta] = house(x)$

Compute $\mathbf{v}$ and β such that $\mathbf{H} = \mathbf{I} - \beta \mathbf{v}.\mathbf{v}^T$ is orthogonal, $v(1) = 1$ and $\mathbf{Hx} = \|\mathbf{x}\|_2 e_1$
$v(1)$ is not stored (known value). We need a vector of size n for β.

Algorithm

Let $\mathbf{A} \in M_{m,n}(\mathbb{R})$ $m \geq n$, be a rectangular matrix.
The step computing the Householder matrices reads

for $k = 1,n$ **do**
  $[\mathbf{v}, \beta] = house(\mathbf{A}(k:m,k))$
  $\mathbf{A}(k:m, k+1:n) = (\mathbf{I}_{m-k+1} - \beta \mathbf{v}.\mathbf{v}^T)\mathbf{A}(k:m; k+1:n)$
  $\mathbf{A}(k,k) = -sign(\mathbf{A}(k,k))\|\mathbf{A}(k:m,k)\|$
  if $k \leq m$ **then**
    $\mathbf{A}(k+1:m,k) = \mathbf{v}(2:m-k+1)$
  end if
end for

Computational complexity of the **QR** factorization

Step factorizing $\mathbf{A} = \mathbf{QR}$ with $\mathbf{A} \in M_{m,n}(\mathbb{R})$

for $k = 1,n$ **do**
(1) Compute β (inner product)
  for $j = 1,n-k$ **do**
    /* Let Col_j be the jth column of $\mathbf{A}_k$ (size $m-k$)*/
(2) $tmp = \mathbf{v}_k^{\mathrm{T}} \times Col_j$ ($\mathbf{v}_k \in M_{m-k}(\mathbb{R})$)
(3) $tmp = \beta \times tmp$
(4) $Col_j = Col_j - tmp \times \mathbf{v}_k$
  end for
end for

$2(m-k)$ flops for step (2) and (4), so $4(m-k)(n-k)$ flops per step k.
It results in

$$4 \sum_{k=1}^{n} (m-k)(n-k) \simeq 4 \left(n^3/3 + (m-n)n(n-1)/2 \right)$$

It leads to a computational complexity of $\frac{4}{3}n^3$ for square matrices of size n

Givens rotations

- 2×2 rotation:

$$G(\theta) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \text{ orthogonal}$$

with $c = cos(\theta)$ and $s = sin(\theta)$.

- Application: $x = \{x_1, x_2\}$

$$c = \frac{x_1}{(x_1^2 + x_2^2)^{\frac{1}{2}}} \text{ et } s = \frac{-x_2}{(x_1^2 + x_2^2)^{\frac{1}{2}}}$$

$$y = (y_1, y_2) = G^T x \text{ then } y_2 = 0$$

- Specify the entries of a matrix that will vanish.

Givens rotations

- Example: QR factorization

$$A = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \\ v_1 & v_2 & v_3 \end{bmatrix}$$

- We find (c, s) such that :

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix}^T \times \begin{bmatrix} r_{11} \\ v_1 \end{bmatrix} = \begin{bmatrix} r'_{11} \\ 0 \end{bmatrix}$$

Givens rotations

- Rotation in the (1,4) plane:

$$G(1,4) = \begin{bmatrix} c & 0 & 0 & s \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -s & 0 & 0 & c \end{bmatrix}$$

$$G(1,4)^T \times A = \begin{bmatrix} r'_{11} & r'_{12} & r'_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \\ 0 & v'_2 & v'_3 \end{bmatrix}$$

- Computational complexity: $2n^3$ flops for the triangularization of the matrix.

TP1 – Variantes du schéma d'orthogonalisation de Gram-Schmidt

Soit $A \in \mathbb{R}^{n \times m}$ avec $n \geq m$ une matrice de rang plein m. Le procédé d'orthonormalisation de Schmidt permet de construire une base orthogonale Q de l'espace image de A. Ce procédé consiste à parcourir les colonnes de A et pour chaque colonne générer un colonne de Q. Chaque nouvelle colonne de Q est obtenue en orthogonalisant la colonne courante de A par rapport aux colonnes précédemment calculées de Q puis à normaliser le vecteur ainsi obtenu. Du point de vue de la mise en œuvre sur un ordinateur, deux implantations peuvent être envisagées.

La variante dite de Gram-Schmidt classique consiste à orthogonaliser "simultanément" chaque colonne de A par rapport aux colonnes de Q déjà calculées. La description algorithmique est la suivante :

1: **for** toutes les colonnes de A **do**
2: Calculer les composantes de la colonne courante de A suivant les directions des colonnes de Q précédemment calculées
3: Retrancher à la colonne courante de A ses composantes suivant les directions des colonnes de Q précédemment calculées
4: Normaliser le vecteur obtenu pour construire une nouvelle colonne de Q
5: **end for**

Cet algorithme peut être reformulé en ne réalisant pas "simultanément" les orthogonalisations mais en les sérialisant. La description algorithmique de cette variante, connue sous le nom d'algorithme de Gram-Schmidt modifié, s'écrit alors :

1: **for** toutes les colonnes de A **do**
2: Copier la colonne courante de A dans un vecteur y
3: **for** toutes les colonnes précédemment calculées de Q **do**
4: Calculer la composante de y suivant la colonne courante de Q
5: Retrancher à y sa composante suivant la direction de la colonne courante de Q
6: **end for**
7: Normaliser y, le résultat constitue la nouvelle colonne de Q
8: **end for**

Ces deux algorithmes sont rigoureusement identiques en arithmétique exacte, mais ont des comportements différents sur des ordinateurs en arithmétique finie. L'objectif de ce TP est d'étudier la perte d'orthogonalité dans les bases générées par ces deux algorithmes pour des matrices dont le conditionnement

croît. La perte d'orthogonalité sera mesurée par

$$\|I - Q^T Q\|;$$

on notera que comme pour tout estimateur en arithmétique finie, il s'agit d'une quantité relative ; ici le dénominateur est $\|I\| = 1$ qui n'apparaît donc pas.

1. Ecrire une fonction Matlab `mgs` qui a pour paramètre d'entrée la matrice A et qui renvoie la matrice Q calculée par la méthode de Gram-Schmidt modifiée.
2. Ecrire une fonction Matlab `cgs` qui a pour paramètre d'entrée la matrice A et qui renvoie la matrice Q calculée par la méthode de Gram-Schmidt classique.
3. Modifier le script Matlab `tp.m` afin de générer une séquence de matrices A_k, $k = 1, \ldots, 16$, dont le conditionnement est $\kappa(A_k) = 10^k$.
   Lancer le script `tp.m` pour représenter sur un même graphique les valeurs de $\|I - Q_k^T Q_k\|$ en fonction de k pour chacun de ces deux schémas d'orthonormalisation et pour chacune de ces matrices A_k. Conjecturer la dépendance de la perte d'orthogonalité en fonction du conditionnement.
4. On considère maintenant des schémas d'orthonormalisation qui consistent à itérer deux fois chacun des deux schémas précédents. Modifier le script Matlab `tp.m` pour réaliser la question précédente pour ces deux schémas (on visualisera sur le même graphique les quatre courbes).

2 Décomposition en valeurs singulières et produits.

Soit A une matrice rectangulaire de $\mathbb{R}^{m\times n}$ de rang $r > 0$. On appelle $A^\dagger$ la matrice dite pseudo-inverse de A.

1. Soient $x \in \mathbb{R}^{n\times 1}$ et $y \in \mathbb{R}^{n\times 1}$, tels que $x^T y \neq 0$. Calculer $(xy^T)^\dagger$ et $(y^T)^\dagger x^\dagger$. A-t-on selon vous pour toutes matrices A et B quelconques $(AB)^\dagger = B^\dagger A^\dagger$?

Solution:

$(xy^T) \in \mathbb{R}^{n\times n}$. Pour calculer la pseudo-inverse nous pouvons utiliser la SVD réduite.

Pour $A \in \mathbb{R}^{m\times n}$, avec $\texttt{rank}(A) = r > 0$, la SVD réduite est $A = U\Sigma V^T$ avec $U \in \mathbb{R}^{m\times r}$, $\Sigma \in \mathbb{R}^{r\times r}$, $V \in \mathbb{R}^{r\times n}$. D'ici nous pouvons obtenir la pseudoinverse de A comme $A^\dagger = V\Sigma^{-1}U^T$.

Dans notre cas $r = 1$ et

$$xy^T = \|x\|\|y\| \frac{x}{\|x\|} \frac{y^T}{\|y\|} \tag{1}$$

donc $U = \frac{x}{\|x\|}$, $V = \frac{y^T}{\|y\|}$ et $\Sigma = \|x\|\|y\|$ et donc $(xy^T)^\dagger = \frac{1}{\|x\|\|y\|} \frac{y}{\|y\|} \frac{x^T}{\|x\|}$.

Pour un vecteur $x \in \mathbb{R}^n$ non nul, la pseudo-inverse est définie comme suit:

$$x^\dagger = \frac{x^T}{\|x\|^2} \tag{2}$$

Donc

$$(y^T)^\dagger x^\dagger = \frac{y}{\|y\|^2} \frac{x^T}{\|x\|^2}. \tag{3}$$

Dans ce cas $(xy^T)^\dagger = (y^T)^\dagger x^\dagger$, mais en général ceci n'est pas vrai pour toutes matrices A,B. Par exemple pour $A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ et $B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ l'on peut verifier que $(AB)^\dagger \neq B^\dagger A^\dagger$.

Dans les quatre cas suivantes l'égalité $(AB)^\dagger = B^\dagger A^\dagger$ est vraie:

1. *$A^T A = I$*
2. *$BB^T = I$*
3. *les colonnes de A et les lignes de B sont indépendantes*
4. *$B = A^T$*

2. Soit A une matrice rectangulaire de $\mathbb{R}^{m\times n}$ de rang r. Soit la matrice par blocs

$$B = \begin{bmatrix} A \\ A \end{bmatrix} \in \mathbb{R}^{2m\times n}.$$

Montrer que $B^\dagger = \alpha \left[A^\dagger, A^\dagger\right] \in \mathbb{R}^{n\times 2m}$, où α est un réel que vous indiquerez.

Solution:

Pour prouver que $B^\dagger = \alpha[A^\dagger, A^\dagger]$ il faut vérifier les quatre propriétés suivantes qui caractérisent la pseudo-inverse:

1. *$AA^\dagger A = A$*
2. *$A^\dagger AA^\dagger = A^\dagger$*
3. *$(AA^\dagger)^T = AA^\dagger$*
4. *$(A^\dagger A)^T = A^\dagger A$*

Par exemple pour la première:

$$\alpha \begin{bmatrix} A \\ A \end{bmatrix} [A^\dagger, A^\dagger] \begin{bmatrix} A \\ A \end{bmatrix} = 2\alpha \begin{bmatrix} A \\ A \end{bmatrix} A^\dagger A = \alpha \begin{bmatrix} AA^\dagger A \\ AA^\dagger A \end{bmatrix} = 2\alpha \begin{bmatrix} A \\ A \end{bmatrix} \tag{4}$$

Pour avoir la première propriété vérifiée il faut $\alpha = 1/2$.

On vérifie de même les autres propriétés.

3. Soit A une matrice rectangulaire de $\mathbb{R}^{m\times n}$ de rang r. On suppose que $A = FR^T$ où $F \in \mathbb{R}^{m\times r}$ et $R \in \mathbb{R}^{n\times r}$. Expliquer pourquoi F et R sont de rang r. Montrer que R^TR et F^TF sont inversibles. Montrer que la pseudo-inverse de A est la matrice $A^\dagger = R(R^TR)^{-1}(F^TF)^{-1}F^T$.

On suppose $A \in \mathbb{R}^{m\times n}$, $A = FR^T$, $R \in \mathbb{R}^{n\times r}$, $F \in \mathbb{R}^{m\times r}$, $r = \texttt{rank}(A)$.

$$r = \texttt{rank}(A) = \texttt{rank}(FR^T) \leq \min\{\texttt{rank}(F), \texttt{rank}(R^T)\} = \min\{\texttt{rank}(F), \texttt{rank}(R)\} \leq r \tag{5}$$

Alors $\texttt{rank}(F) = \texttt{rank}(R) = r$.

$R^TR, F^TF \in \mathbb{R}^{r\times r}$ *et*

$$\texttt{rank}(R^TR) = \texttt{rank}(R) = r \tag{6}$$

(En effet $ker(R^TR) = ker(R)$: $\supset$ évident et si x est tel que $R^TRx = 0$ alors $x^TR^TRx = 0$ d'ou $||Rx|| = 0$ donc $Rx = 0$ et $x \in ker(R)$. On conclut avec le théorème du rang : $r = dimker(R^TR) + rang(R^TR) = dimker(R) + rang(R))$[1]. Donc R^TR est inversible (idem pour F^TF).

Pour prouver que $A^\dagger = R(R^TR)^{-1}(F^TF)^{-1}F^T$, on passe par la caractérisation de Moore-Penrose (cf question précédente).

Par exemple, on a pour la première propriété :

$$AA^\dagger A = FR^TR(R^TR)^{-1}(F^TF)^{-1}F^TFR^T = FR^T = A \tag{7}$$

On vérifie de même les autres propriétés.

3 Factorisation QR

1. On suppose que l'on applique l'algorithme de la factorisation QR de Householder présenté en cours à une matrice A carrée de taille n qui possède deux colonnes j_1 et j_2 ($j_1 < j_2$) colinéaires.

   (a) Montrer qu'à chaque étape k de l'algorithme QR, les colonnes j_1 et j_2 de la matrice mise à jour $H_k \dots H_1 A$ restent colinéaires.

   Solution:

   A chaque étape on effectue un opération orthogonale de réflexion sur chacune des colonnes; cette opération préserve la colinéarité (on a surement besoin d'hypothèse moins forte)

   (b) On s'intéresse à l'étape j_1 de la factorisation QR. Que se passe-t-il pour le colonne j_2? On précisera les éléments non nuls de cette colonne.

   Solution:

   Tous les éléments de la colonne j_2 dans la matrice réduite A_{j_1} d'ordre $n - j_1$ sont nuls. En effet la colonne j_2 reste toujours colinéaire à la colonne j_1 et comme tous les éléments sous la diagonale de la colonne de j_1 ont été mis à 0 il en est nécessairement de même pour tous les éléments sous la ligne j_1 de la colonne j_2.

   (c) En déduire une modification de l'algorithme de la factorisation QR qui permette de détecter un ensemble de vecteurs colonnes colinéaires aux précédents et donc la dimension du noyau (espace des vecteurs v tels que $Av = 0$) de la matrice carrée A. Indiquer comment la dimension du noyau est calculée en fin de la factorisation QR.

[1] Si on avait $rang(R) = n$ ce serait RR^T qui serait inversible. Ne pas oublier que dans le théorème du rang c'est la dimension de l'espace de départ qui apparait, qui n'est pas le même pour RR^T et R^TR

Solution:
Par analogie avec la factorisation LU avec pivotage il faut permuter les colonnes pour finir avec un bloc de 0 et une structure similaire à celle de l'exercice 2 avec un bloc S nul. du coup ils ont construit une factorisation QR avec pivotage des colonnes qui permet d'obtenir

$$AP = QR = \begin{pmatrix} Q_1 & Q_2 \end{pmatrix} \begin{pmatrix} R_{11} & R_{21} \\ & 0 \end{pmatrix}$$

2. Soit $A \in M_{n,n}(\mathbb{R})$. On suppose de plus qu'il existe une matrice de permutation P des colonnes de A telle qu'après k étapes de la factorisation QR de A on obtient la décomposition suivante:

$$AP = QR = \begin{pmatrix} Q_1 & Q_2 \end{pmatrix} \begin{pmatrix} R_{11} & R_{21} \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} Q_1 & Q_2 \end{pmatrix} \begin{pmatrix} R_1 \\ 0 \end{pmatrix}$$

avec $Q_1 \in M_{n,k}(\mathbb{R})$ telle que $Q_1^T Q_1 = I_k$, $Q_2 \in M_{n,n-k}(\mathbb{R})$ telle que $Q_2^T Q_2 = I_{n-k}$, $R_{11} \in M_{k,k}(\mathbb{R})$ une matrice triangulaire inversible, $R_{12} \in M_{k,n-k}(\mathbb{R})$ et $R_1 \in M_{k,n}(\mathbb{R})$

   (a) Soit A' la matrice permutée $A' = AP$, montrer que les colonnes de Q_1 forment une base de l'espace engendré par les colonnes de A' et en déduire le rang de A' (dimension de l'espace engendré par les colonnes de A'). (On notera que P étant une matrice de permutation on peut en déduire que les espaces engendrés par les colonnes de A' et de A sont identiques).
   Solution:
   Soit A' la matrice permutée,

$$A' = AP = Q_1 R_1 = \begin{pmatrix} A'_1 & A'_2 \end{pmatrix}$$

   et donc

$$a'_{ij} = \sum_{l=1}^{k} q_{il} r_{lj}$$

   On peut en déduire que pour toute colonne j de la matrice permutée A':

$$\textit{Colonne } j \textit{ de } A' = \sum_{l=1}^{k} r_{lj} \times \textit{Colonne } l \textit{ de } Q_1$$

   Toute colonne j de A' est donc une combinaison linéaire des k colonnes de Q_1. Donc $dim(Vect(A')) \leq k$. Par ailleurs R_{11} étant triangulaire inversible on a donc pour toute colonne $j \leq k$ de A' que la colonne j est combinaison linéaire des j premières colonnes de Q_1.

$$\textit{Colonne } j \textit{de } A' = \sum_{l=1}^{j} r_{lj} \times \textit{Colonne } l \textit{ de } Q_1$$

   Les k premières colonnes de A' sont donc linéairement indépendantes et $dim(Vect(A')) \geq k$.
   Et donc $dim(Vect(A')) = k$.

   (b) Montrer que $(AP)^T \times Q_2 = 0$ (i.e. Q_2 est orthogonal à l'espace engendré par les colonnes de AP).
   Solution:

$$(AP)^T \times Q_2 = \begin{pmatrix} R_1^T & 0 \end{pmatrix} \begin{pmatrix} Q_1^T \\ Q_2^T \end{pmatrix} \times Q_2 = \begin{pmatrix} R_1^T & 0 \end{pmatrix} \begin{pmatrix} Q_1^T Q_2 = 0 \\ Q_2^T Q_2 = I_k \end{pmatrix} = \begin{pmatrix} R_1^T \times 0 & 0 \times I_k \end{pmatrix} = 0$$

3. Soit $A \in M_{n,n}(\mathbb{R})$. On suppose qu'à l'issue d'une factorisation QR on obtient la décomposition suivante:

$$A = QR = \begin{pmatrix} Q_1 & Q_2 \end{pmatrix} \begin{pmatrix} R_1 \\ R_2 \end{pmatrix}$$

avec $Q_1 \in M_{n,k}(\mathbb{R})$, $Q_2 \in M_{n,n-k}(\mathbb{R})$, $R_1 \in M_{k,n}$ et $R_1 \in M_{n-k,n}$
On suppose de plus que $\|R_2\| \leq \epsilon \|A\|$

   (a) Montrer que $\|A - Q_1 R_1\| \leq \epsilon \|A\|$
   (b) On suppose que la matrice A est approximée par la matrice $Q_1 R_1$. Pour quelles valeurs de k cette approximation permet elle de réduire l'espace mémoire nécessaire pour stocker la matrice A par un facteur 10 ?

Solution:
stockage de Q_1 et R_1 : $k \leq n/20$

4 Calcul de l'inverse d'une matrice

On suppose que A est une matrice carrée de taille n inversible pour laquelle existe une factorisation LU sans pivotage de la matrice A:

$$A = LU$$

1. Montrer que l'on peut utiliser l'expression suivante $AA^{-1} = I$ et la décomposition $A = LU$ pour calculer A^{-1}.
   On détaillera les étapes du calcul de A^{-1}.

   Solution:
   $L(UA^{-1}) = I$

   (a) $LY = I$ avec Y matrice carrée d'ordre n, n résolutions de systèmes linéaires
   (b) $UA^{-1} = U$, n résolutions de systèmes linéaires

2. Calculer la complexité en terme de nombre d'opérations flottantes du calcul de A^{-1} de la question précédente. On prendra en compte aussi le nombre d'opérations pour la factorisation LU de A.

   Solution:
   $2/3n^3 + n * n^2 + n * n^2$

3. Soit x la solution du système $Lx = e_j$, avec e_j le j^{ieme} vecteur de la base canonique (vecteur possédant un seul élément non nul égal à 1 en position j). Montrer que les $(j-1)$ premières composantes du vecteur x sont nulles : $\forall i \in \mathbb{N}, 1 \leq i \leq j-1, \quad x_i = 0$.

   Solution:
   V0: on peut le montrer avec les mains/pieds en déroulant les $j-1$ premières étapes de la résolution
   V1: on décompose L en bloc L_{11} avec les $j-1$ premières lignes, L_{21} et L_{22} les lignes restantes

$$\begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix}$$

   l'equation associée aux j premières lignes $L_{11}x_1 = 0$ implique que x_1 est un vecteur nul car L_{11} est inversible, triangulaire avec des 1 sur la diagonale, déterminant égal à 1.

4. En déduire le nombre d'opérations pour calculer $Lx = e_j$, où e_j est le j^{ieme} vecteur de la base canonique.

   Solution:
   résolution du système triangulaire de taille $n - j + 1$ *et donc* $(n - j + 1)^2$

5. En déduire le coût de calcul de $LX = I$. On pourra noter que $X = L^{-1}$. On rappelle que $\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$.

   Solution:
   $\sum_{j=1}^{n}(n - j + 1)^2 = 1/3n^3$

6. En déduire le coût réel du calcul complet de $AA^{-1} = I$

   Solution:
   $2/3n^3 + 1/3n^3 + n * n^2 = 2n^3$